

Resumen

Este Trabajo de Fin de Grado combina el desarrollo de una aplicación web con el uso de técnicas de aprendizaje automático, o *machine learning* en inglés, aplicadas al análisis y predicción de datos orientados a la gestión de inventario y a la estrategia de negocio de una frutería. El objetivo principal ha sido dotar al negocio de una herramienta que permita analizar valores y métricas clave para optimizar la toma de decisiones en el día a día. Para alcanzar este objetivo, se han empleado herramientas de visualización de datos junto con técnicas de aprendizaje automático como **clustering**, **regresión lineal** y redes neuronales del tipo **LSTM** (*Long Short-Term Memory*), cada una con un propósito específico dentro del análisis.

El *clustering* ha sido utilizado para agrupar productos en función de características comunes, facilitando así la comprensión de patrones de comportamiento en el inventario. Por otro lado, la regresión lineal y los modelos *LSTM* se han aplicado a la predicción de inventario futuro usando series temporales, permitiendo anticipar la demanda y mejorar la planificación. Todo el trabajo se ha desarrollado desde una perspectiva investigadora, incluyendo comparativas entre los modelos utilizados, análisis de resultados y evaluación de su precisión mediante distintas métricas. De este modo, el proyecto no solo ofrece una **solución práctica**, sino que también contribuye al estudio de técnicas de predicción en entornos de negocio reales.

Se ha desarrollado una **aplicación web funcional** con un diseño minimalista que integra tres apartados: un panel de control con métricas y visualizaciones sobre el registro de ventas del negocio, un apartado donde se puede hacer uso de *deep learning* para predecir las ventas de algunos productos y por último un apartado donde se hace uso del algoritmo de *clustering* y se recogen las agrupaciones de productos junto con su caracterización.

Índice general

I	Introducción	1
1.	Contexto	1
2.	Motivación y objetivos	1
3.	Estructura del documento	2
4.	Herramientas y entorno de desarrollo	3
4.1.	Herramientas	3
4.2.	Entorno de desarrollo y herramientas utilizadas	3
II	Estado del arte	5
1.	El Mundo Empresarial	5
2.	Series Temporales	5
3.	Aprendizaje Automático	7
3.1.	Aprendizaje supervisado y no supervisado	7
3.2.	Regresión Lineal	9
3.3.	XGBoost	11
3.4.	Clustering	12
4.	Aprendizaje Profundo	17
4.1.	Unidad de procesamiento: neurona	17
4.2.	Perceptrón Multicapa (MLP)	18
4.3.	Red neuronal recurrente (RNN)	19
4.4.	Long Short Term Memory (LSTM)	21
III	Planificación y costes	25
1.	Documento de Requisitos	25
1.1.	Introducción	25
1.2.	Requisitos Generales	26
1.3.	Requisitos de información	28
1.4.	Requisitos funcionales	30
1.5.	Reglas de negocio	32
1.6.	Requisitos no funcionales	34
1.7.	Matriz de trazabilidad	35
2.	Alcance del proyecto	36
2.1.	Descripción del alcance	36
2.2.	Exclusiones	37
2.3.	Restricciones	38
2.4.	Supuestos	40
3.	Hitos del proyecto	41
4.	Estructura de Descomposición del Trabajo (EDT)	43
4.1.	Diccionario EDT	44
5.	Actividades y estimaciones	47
5.1.	Estimación de Recursos	47
5.2.	Actividades	48

6.	Cronograma	53
7.	Desviación de la planificación	56
8.	Metodología de desarrollo	56
8.1.	Comparación de metodologías de gestión de proyectos	56
8.2.	Metodología utilizada	57
IV	Ejecución	59
1.	Análisis exploratorio de los datos (EDA)	59
1.1.	Búsqueda y elección del conjunto de datos	59
1.2.	Carga y visualización inicial de los datos	59
1.3.	Preparación de los datos	60
1.4.	Exploración inicial del dataset unificado	61
1.5.	Análisis de distribución y relaciones entre variables	63
1.6.	Visualización de las métricas	68
1.7.	Conclusiones EDA	72
2.	Desarrollo de la aplicación	73
2.1.	Arquitectura y diseño de la interfaz	73
2.2.	Manual de Usuario	75
2.3.	Desarrollo del Backend	75
2.4.	Desarrollo del Frontend	79
3.	Investigación y Predicción	83
3.1.	Metodología y Enfoque Experimental	84
3.2.	Clustering	92
V	Conclusión	98
	Bibliografía	99

Índice de figuras

1.	Ejemplo Serie Temporal - IPC 1990-2011.	7
2.	Simple Linear Relationship	10
3.	Lineal - No lineal	11
4.	Homoscedasticidad	11
5.	K-medias	13
6.	Método del Codo	14
7.	Clustering Jerárquico	15
8.	Aglomerativo - Divisivo	16
9.	DBSCAN	17
10.	Neurona Artificial	18
11.	Funciones de activacion ANN's	18
12.	Ejemplo red MLP con dos capas ocultas.	19
13.	RNN - FNN.	20
14.	Esquema de una celda RNN en el tiempo.	20
15.	Problema del gradiente desvanecido.	21
16.	Celda LSTM.[1]	22
17.	Tareas de elicitación de requisitos	25
18.	Cuadro de Requisitos Generales	27
19.	Matriz de trazabilidad	36
20.	Hito 01	41
21.	Hito 02	41
22.	Hito 03	42
23.	Hito 04	42
24.	Hito 05	42
25.	Hito 06	43
26.	Hito 07	43
27.	Estructura de Descomposición del Trabajo (EDT)	44
28.	Paquete de trabajo 1.X: Documentos iniciales	44
29.	Paquete de trabajo 2.X: Identificación de requisitos	44
30.	Paquete de trabajo 3.X: Planificación del proyecto	45
31.	Paquete de trabajo 4.X: Preparación de los datos	45
32.	Paquete de trabajo 5.X: Desarrollo del sistema	45
33.	Paquete de trabajo 6.X: Backend	45
34.	Paquete de trabajo 7.X: Frontend	46
35.	Paquete de trabajo 8.X: Modelos ML	46
36.	Paquete de trabajo 9.X: Gestión del desarrollo	46
37.	Paquete de trabajo 10.X: Memoria	46
38.	Paquete de trabajo 11.X: Entrega Final	47
39.	Cuadro de recursos humanos	47
40.	Cuadro de recursos físicos	48
41.	Cuadro actividades paquete de trabajo 1.X	48

42.	Cuadro actividades paquete de trabajo 2.X	49
43.	Cuadro actividades paquete de trabajo 3.X	49
44.	Cuadro actividades paquete de trabajo 4.X	50
45.	Cuadro actividades paquete de trabajo 5.X	50
46.	Cuadro actividades paquete de trabajo 5.1.X	51
47.	Cuadro actividades paquete de trabajo 5.1.X	51
48.	Cuadro actividades paquete de trabajo 5.1.X	52
49.	Cuadro actividades paquete de trabajo 6.X	52
50.	Cuadro actividades paquete de trabajo 7.X	53
51.	Cuadro actividades paquete de trabajo 8.X	53
52.	Diagrama de Gantt	55
53.	Panel de control Kanban	58
54.	Resultado carga y visualización de los datos	60
55.	Resultado Preparación de los datos	61
56.	Resultado exploración del DataFrame unido	63
57.	Resultado análisis de variables categóricas y numéricas	65
58.	Resultado correlación y creación de métricas derivadas	67
59.	Histograma de la cantidad vendida (kilo)	68
60.	Histograma del precio unitario en EUR/kg	69
61.	Relación entre cantidad vendida y precio	70
62.	Ingresos totales por categoría	71
63.	Evolución diaria de los ingresos	72
64.	Arquitectura del sistema	73
65.	Diseño de la página principal de la aplicación web	74
66.	Diseño de la página de previsión de inventario de la aplicación web	74
67.	Diseño de la página de grupos de productos de la aplicación web	75
68.	Resultados de la regresión univariante	86
69.	Resultados de la regresión bivariada	87
70.	Resultados de la regresión multivariante	88
71.	Resultados del modelo LSTM univariante	89
72.	Resultados del modelo LSTM multivariante	89
73.	Resultados del modelo LSTM bivariado	90
74.	Matriz transacción-producto.	93
75.	Regla del codo para la selección de k	94
76.	Proyección t-SNE para $k = 13$	95
77.	Proyección t-SNE para $k = 5$	95
78.	Proyección t-SNE para $k = 7$	96

Índice de tablas

IV.1. Comparación de rendimiento de modelos: MAE y RMSE	90
IV.2. Representantes por cluster (top-5 productos por peso en el centroide).	96

Índice de extractos de código

IV.1. Carga y visualización de los datos	60
IV.2. Preparación de los datos	60
IV.3. Exploración del DataFrame unido	62
IV.4. Análisis de variables categóricas y numéricas	64
IV.5. Correlación y creación de métricas derivadas	66
IV.6. Histograma de cantidad vendida	68
IV.7. Histograma del precio de venta unitario	69
IV.8. Scatterplot cantidad vs precio	69
IV.9. Ingresos por categoría	70
IV.10. Evolución de ingresos diarios	71

I. Introducción

1. Contexto

En la actualidad, uno de los pilares fundamentales en el mundo empresarial es la capacidad de interpretar grandes cantidades de datos mediante herramientas especializadas. Este proceso supone un gasto significativo para las empresas, por lo que es imprescindible que los beneficios obtenidos justifiquen plenamente dicha inversión.

Con frecuencia, esto lleva a pensar que estas tecnologías no están al alcance de todas las empresas, o que simplemente no compensan el esfuerzo y coste asociados. No obstante, pequeños negocios como una frutería también pueden aprovechar estas herramientas. Por ejemplo, una mejor gestión del inventario puede marcar una diferencia significativa en rentabilidad y eficiencia operativa. Anticipar la demanda o reducir la compra innecesaria de productos perecederos son solo algunos beneficios claros derivados del uso de técnicas de análisis y predicción.

A pesar de estos beneficios, integrar soluciones tecnológicas avanzadas en pequeñas y medianas empresas (*pymes*) no es tarea sencilla. Según el ONTSI [2], la brecha tecnológica entre *pymes* y grandes empresas sigue existiendo. Mientras las *pymes* enfrentan dificultades relacionadas con la falta de personal especializado, recursos financieros limitados y carencia de estrategias claras de adopción tecnológica, las grandes empresas tienden a seguir un enfoque planificado y estructurado. Esto último puede generar un efecto de arrastre que impulse la digitalización de las *pymes*.

Este trabajo busca abordar no solo el problema específico de la gestión de inventario en una frutería, sino también mejorar el acceso a herramientas clave como el aprendizaje automático. Además, explora diferentes técnicas de aprendizaje automático aplicadas a casos reales, aportando tanto una visión práctica como una comparación sobre su eficacia.

2. Motivación y objetivos

El objetivo principal de este proyecto es aplicar técnicas propias de la ciencia de datos en un caso real que presenta ciertas particularidades debido a su naturaleza. La gestión de inventario dentro de una frutería, desde un punto de vista analítico, es especialmente interesante debido a la rápida rotación de productos, su carácter perecedero y la sensibilidad a factores externos como la estacionalidad o las condiciones climáticas, lo que implica que anticiparse a la demanda no sea una tarea sencilla.

La motivación de este proyecto aparece de la necesidad de demostrar que estas técnicas analíticas avanzadas pueden ayudar a pequeños negocios y cotidianos, no solo a grandes empresas y corporaciones.

Para ello, se plantean una serie de objetivos específicos:

- Analizar el comportamiento del stock a partir de datos históricos.
- Desarrollar modelos predictivos que permitan anticipar la demanda futura.
- Visualizar de forma clara y comprensible las métricas clave del inventario.
- Evaluar diferentes algoritmos de *machine learning* en cuanto a su rendimiento y aplicabilidad.
- Integrar los resultados en una herramienta práctica que sirva de apoyo en la toma de decisiones del negocio.

En definitiva, este TFG pretende ser una muestra de cómo los datos pueden convertirse en una herramienta estratégica, incluso en entornos sencillos, y cómo el conocimiento técnico adquirido a lo largo del grado puede aplicarse para resolver problemas reales de forma útil y tangible.

3. Estructura del documento

El documento para la memoria está dividido en cinco capítulos, a través de los cuales se desarrolla todo el trabajo realizado durante el proyecto. La estructura principal del documento es la siguiente:

- **Capítulo I: Introducción.** Este capítulo presenta el contexto del proyecto, los objetivos que se persiguen y la motivación personal que ha llevado a su desarrollo. También aborda la estructura del documento en sí y las herramientas y el entorno de desarrollo utilizados.
- **Capítulo II: Estado del arte.** Se realiza una revisión de los trabajos, técnicas y herramientas existentes que están relacionados con el problema abordado. Cubre temas como el mundo empresarial, series temporales, aprendizaje automático (supervisado y no supervisado, regresión lineal, clustering) y aprendizaje profundo (neuronas artificiales, MLP, RNN, LSTM).
- **Capítulo III: Planificación y costes.** Detalla los aspectos de la planificación del proyecto. Incluye el documento de requisitos (generales, de información, funcionales, reglas de negocio, no funcionales, matriz de trazabilidad), el alcance del proyecto (descripción, exclusiones, restricciones, supuestos), hitos, la Estructura de Descomposición del Trabajo (EDT), actividades y estimaciones, cronograma y la metodología de desarrollo Kanban seguida.
- **Capítulo IV: Ejecución.** Describe la implementación práctica del proyecto. Esto abarca el análisis exploratorio de los datos (EDA), incluyendo la búsqueda y elección del conjunto de datos, carga, preparación, exploración inicial, análisis de distribución y visualización de métricas. También se detalla

el desarrollo de la aplicación (arquitectura, diseño de la interfaz) y la investigación y predicción (evaluación y comparación de modelos).

- **Capítulo V: Conclusión.** En este capítulo final se resumen los resultados obtenidos, se reflexiona sobre el proceso y se proponen posibles líneas de mejora o continuidad del proyecto.

4. Herramientas y entorno de desarrollo

4.1. Herramientas

El proyecto se divide en tres partes bien diferenciadas: el **frontend**, el **backend** y el **módulo de modelado predictivo**.

En primer lugar, el **frontend** se encarga de toda la interfaz de la aplicación web, desarrollada con React mediante *Create React App* [3]. Esta capa permite al usuario interactuar con el sistema de forma visual e intuitiva, consultando las métricas clave y visualizando las predicciones generadas a partir de los datos históricos.

En segundo lugar, el **backend**. Esta parte se ha desarrollado en Python utilizando el microframework *Flask* [4], y expone una API REST a través de la cual se gestionan las peticiones del cliente. Además, se ha integrado el paquete *Flask-CORS* para permitir la comunicación entre el servidor y el cliente alojados en distintos orígenes. Por otro lado, para las gráficas y visualizaciones se utiliza la librería de código abierto *bokeh* [5] que permite añadir una capa extra de interactividad.

Por último, el módulo de **modelado predictivo** es la parte del proyecto donde se comparan diferentes algoritmos de aprendizaje automático y donde se lleva a cabo la parte experimental del proyecto. Esta parte del proyecto también se ha desarrollado en Python, empleando diversas librerías de ciencia de datos y aprendizaje automático, como *pandas*, *scikit-learn*, *xgboost* y *keras*. Los modelos se entrenan a partir de los datos históricos del inventario y se integran con el backend para ofrecer predicciones en tiempo real a la interfaz.

4.2. Entorno de desarrollo y herramientas utilizadas

En cuanto al entorno de desarrollo, el proyecto ha sido llevado a cabo utilizando tres herramientas principales: **GitHub Desktop**, **Visual Studio Code** y **Jupyter Notebook**.

GitHub Desktop ha sido utilizado como plataforma de control de versiones y repositorio remoto. Gracias a esta herramienta, que facilita el uso de **GitHub** debido a su interfaz intuitiva, ha sido posible mantener un historial claro y ordenado de los cambios realizados a lo largo del desarrollo, así como garantizar la trazabilidad del código y facilitar la colaboración y respaldo del proyecto.

Visual Studio Code ha sido el editor principal para la implementación tanto del backend como del frontend. Su versatilidad, la amplia disponibilidad de extensiones y su integración con sistemas de control de versiones han facilitado la edición eficiente del código, la depuración y la gestión general del proyecto.

Por otro lado, **Jupyter Notebook** se ha empleado durante la fase de análisis exploratorio de datos y desarrollo de los modelos predictivos. Este entorno ha permitido trabajar de forma interactiva con los datos, visualizar métricas de evaluación y documentar los experimentos de forma clara y reproducible, lo cual ha sido clave en el desarrollo iterativo del modelo.

II. Estado del arte

1. El Mundo Empresarial

En los últimos años, la digitalización de los procesos empresariales ha transformado la forma en que las organizaciones toman decisiones. El avance de la ciencia de datos y, en particular, del aprendizaje automático (*machine learning*), ha permitido desarrollar herramientas capaces de analizar grandes volúmenes de información, detectar patrones ocultos y generar predicciones útiles en múltiples contextos. Este progreso no solo ha beneficiado a grandes corporaciones, sino que también empieza a tener impacto en *pymes*, que pueden aprovechar estas tecnologías para optimizar sus operaciones.[6]

Uno de los ámbitos donde el uso del análisis de datos ha demostrado ser especialmente valioso es la gestión del inventario. La posibilidad de anticiparse a la demanda y ajustar las compras en función de previsiones fundamentadas permite reducir pérdidas, minimizar roturas de stock y mejorar la eficiencia logística. En este contexto, diferentes técnicas han sido aplicadas, desde modelos estadísticos tradicionales como ARIMA, hasta algoritmos más complejos como redes neuronales recurrentes (RNN) o modelos basados en árboles de decisión, como *Random Forest* o *XGBoost* [7].

En el ámbito específico del comercio minorista, y especialmente en sectores con productos perecederos como la alimentación, la predicción de la demanda presenta desafíos añadidos. Factores como la estacionalidad, las condiciones climáticas, los días festivos o incluso el comportamiento del consumidor a corto plazo influyen notablemente en la evolución del inventario. Esto ha llevado al desarrollo de enfoques híbridos que combinan técnicas estadísticas con modelos de *machine learning*, capaces de captar tanto la tendencia general como las fluctuaciones más irregulares del consumo [8].

Además, la accesibilidad de herramientas de código abierto y lenguajes como Python ha democratizado el acceso a este tipo de soluciones. Librerías como *pandas*, *scikit-learn* o *TensorFlow* permiten aplicar técnicas avanzadas sin necesidad de grandes infraestructuras, lo cual abre nuevas oportunidades para los pequeños negocios que desean incorporar análisis de datos a su gestión diaria.

2. Series Temporales

Una **serie temporal** (o simplemente una serie) es una secuencia de N observaciones (datos) ordenadas y equidistantes cronológicamente sobre una característica (serie univariante o escalar) o sobre varias características (serie

multivariante o vectorial) de una unidad observable en diferentes momentos. [9] [10] [11]

- Las representaciones matemáticas frecuentes de series temporales univariantes son:

$$y_1, y_2, \dots, y_N; \quad (y_t)_{t=1}^N; \quad (y_t : t = 1, \dots, N),$$

donde y_t es la observación número t ($1 \leq t \leq N$) de la serie y N es el número de observaciones de que consta la serie completa (el **tamaño** o la **longitud** de la serie). Las N observaciones $y_1, y_2, \dots, y_N$ pueden recogerse en un vector columna

$$\mathbf{y} \equiv \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = [y_1, y_2, \dots, y_N]' \text{ de orden } N \times 1.$$

- Las representaciones matemáticas frecuentes de series temporales multivariantes son:

$$\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N; \quad (\mathbf{y}_t)_{t=1}^N; \quad (\mathbf{y}_t : t = 1, \dots, N),$$

donde $\mathbf{y}_t \equiv [y_{t1}, y_{t2}, \dots, y_{tM}]'$ ($M \geq 2$) es la observación n° t ($1 \leq t \leq N$) de la serie, y N es el número de observaciones de que consta la serie completa. Las N observaciones $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$ pueden recogerse en una matriz $\mathbf{Y}$ de orden $N \times M$:

$$\mathbf{Y} \equiv \begin{bmatrix} \mathbf{y}'_1 \\ \mathbf{y}'_2 \\ \vdots \\ \mathbf{y}'_N \end{bmatrix} \equiv \begin{bmatrix} y_{11} & y_{12} & \cdots & y_{1M} \\ y_{21} & y_{22} & \cdots & y_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ y_{N1} & y_{N2} & \cdots & y_{NM} \end{bmatrix},$$

donde y_{tj} es la observación n° t ($1 \leq t \leq N$) sobre la característica o variable n° j ($1 \leq j \leq M$), que es la misma en todo momento t .

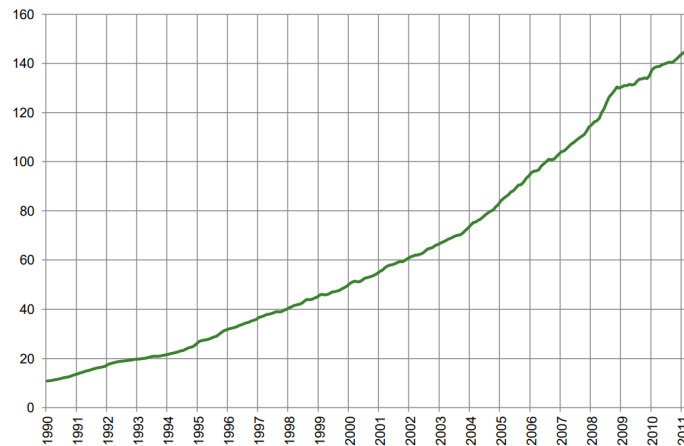


Figura 1: Ejemplo Serie Temporal - IPC 1990-2011.

3. Aprendizaje Automático

“El aprendizaje automático es el campo de estudio que otorga a las computadoras la capacidad de aprender sin ser programadas explícitamente.”

Arthur Samuel, 1959

3.1. Aprendizaje supervisado y no supervisado

El aprendizaje supervisado y el aprendizaje no supervisado son dos de los principales enfoques dentro del aprendizaje automático.

En el **aprendizaje supervisado**, el modelo se entrena utilizando datos etiquetados, es decir, cada entrada está asociada a una salida conocida. El objetivo es que el modelo aprenda a predecir resultados a partir de ejemplos ya conocidos.^[12]

Por otro lado, en el **aprendizaje no supervisado**, el modelo trabaja con datos no etiquetados, lo que le permite descubrir patrones, estructuras o relaciones en los datos sin contar con una salida predefinida.

En esta sección se describen ambos tipos con mayor detalle, ya que son fundamentales para entender las técnicas utilizadas en este trabajo.

Aprendizaje supervisado Como su nombre indica, el aprendizaje supervisado funciona de forma similar a un profesor o supervisor que guía al sistema. En este enfoque, se entrena al modelo utilizando datos etiquetados, es decir, ejemplos que ya contienen la respuesta o categoría correcta asociada a cada entrada. De este modo, el modelo aprende a establecer una relación entre los datos de entrada y sus salidas correspondientes.

Una vez finalizado el entrenamiento, se proporciona al modelo un nuevo conjunto de ejemplos para que, basándose en lo aprendido, sea capaz de predecir de forma correcta el resultado esperado para datos no vistos anteriormente- [13] [14]

Ventajas del aprendizaje supervisado

- Aprende a partir de ejemplos etiquetados, lo que le permite realizar predicciones precisas sobre nuevos datos no vistos previamente.
- A medida que se dispone de más datos y entrenamiento, el modelo puede mejorar su precisión, lo que conlleva un mejor rendimiento y mayor fiabilidad en las predicciones.
- Funciona adecuadamente en múltiples tareas, desde la detección de correos electrónicos no deseados hasta la estimación del precio de viviendas, ya que es capaz de abordar distintos tipos de problemas computacionales.
- Puede utilizarse tanto para tareas de clasificación (asignar categorías) como de regresión (predecir valores numéricos), lo que lo convierte en una herramienta versátil para distintos contextos.

Desventajas del aprendizaje supervisado

- Requiere un conjunto de datos bien etiquetado, en el que cada entrada esté asociada a una salida concreta. La creación de este tipo de conjuntos puede ser costosa en tiempo, dinero y recursos, además de estar sujeta a errores humanos.
- Aunque funciona bien en muchas tareas, puede tener dificultades ante problemas complejos o poco estructurados, como la identificación de patrones abstractos o conceptos que no estén presentes en los datos de entrenamiento.
- Existe el riesgo de *sobreajuste* (overfitting), es decir, que el modelo aprenda demasiado bien los datos de entrenamiento y tenga un rendimiento deficiente frente a nuevos datos.
- Estos modelos suelen necesitar actualizaciones frecuentes con nuevos datos etiquetados para mantener su precisión, ya que los datos del mundo real cambian constantemente.

Aprendizaje no supervisado El aprendizaje no supervisado es una rama del aprendizaje automático que funciona de forma diferente al aprendizaje supervisado, ya que no interviene ningún 'profesor' o guía que indique al sistema cuál es la salida correcta. En este enfoque, se proporciona al modelo un conjunto de datos sin etiquetas ni categorías asignadas.

El sistema analiza los datos por sí mismo con el objetivo de encontrar patrones, estructuras ocultas, grupos o relaciones, sin disponer de información previa sobre

qué debe aprender. Es decir, el modelo aprende organizando los datos internamente, sin conocer de antemano las respuestas correctas.

Por ejemplo, un modelo de aprendizaje no supervisado puede analizar un conjunto de datos sobre animales y agruparlos en función de sus características o comportamientos. Esos grupos, aunque no estén etiquetados explícitamente, pueden corresponder a diferentes especies, lo que permite al sistema clasificar los datos sin necesidad de una supervisión previa. [15] [16]

Ventajas del aprendizaje no supervisado

- No requiere datos etiquetados, lo que permite trabajar con grandes volúmenes de datos de forma más rápida y sencilla desde el inicio.
- Es capaz de manejar grandes conjuntos de datos y reducir su complejidad manteniendo los patrones relevantes, lo que lo hace eficiente y manejable.
- Descubre patrones y relaciones ocultas en los datos que no eran evidentes previamente, proporcionando información valiosa.
- Al analizar datos no etiquetados, permite identificar tendencias y agrupaciones significativas que enriquecen la comprensión del conjunto de datos.

Desventajas del aprendizaje no supervisado

- Al no contar con respuestas etiquetadas, es difícil evaluar con precisión la efectividad del modelo.
- La ausencia de una guía clara puede llevar a resultados menos precisos en problemas complejos.
- Una vez agrupados los datos, puede ser necesario revisar y etiquetar manualmente los grupos obtenidos, lo cual requiere tiempo y esfuerzo.
- La presencia de datos faltantes, valores atípicos o ruido puede afectar negativamente a la calidad de los resultados obtenidos.

3.2. Regresión Lineal

En estadística, la regresión lineal es un modelo matemático utilizado para aproximar la relación de dependencia entre una variable dependiente Y , un conjunto de m variables independientes X_i (donde $m \in \mathbb{Z}^+$) y un término aleatorio ε . Este modelo forma parte de los algoritmos de aprendizaje supervisado [17, 18].

Regresión lineal simple: se relaciona una única variable independiente con una variable dependiente.

$$y = \beta_0 + \beta_1 x_1$$

Regresión lineal múltiple: el modelo de regresión tiene varias variables explicativas y una variable respuesta.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

- y es la variable dependiente.
- x_i es la variable independiente i .
- β_0 es la constante de la ecuación de la regresión lineal múltiple.
- β_i es el coeficiente de regresión asociado a la variable x_i .
- ε es el término de error o residuo, es decir, la diferencia entre el valor observado y el valor estimado por el modelo.
- m es el número total de variables independientes en el modelo.

En este trabajo se han implementado pruebas con ambos tipos por lo que vamos a entrar en detalle en el funcionamiento:

El objetivo de la regresión lineal simple es predecir una variable dependiente a partir de una variable independiente. La precisión depende de la relación lineal entre estas, a mayor relación lineal, mayor precisión a la hora de predecir. Esto va unido al hecho de que cuanto mayor sea la proporción de la varianza de la variable dependiente que puede explicar la variable independiente, más exacta será la predicción. Visualmente, la relación entre las variables puede mostrarse en un diagrama de dispersión. Cuanto mayor sea la relación lineal entre las variables dependiente e independiente, más se situarán los puntos de datos en una línea recta. [19]

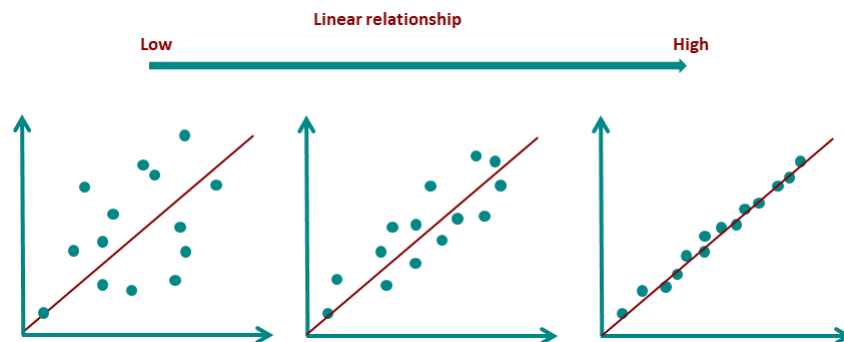


Figura 2: Simple Linear Relationship

En el caso de la regresión lineal múltiple, el modelo permite incorporar varias variables independientes para predecir una única variable dependiente. Esto mejora el poder explicativo del modelo al considerar múltiples factores simultáneamente. Cada variable aporta una contribución ponderada (representada por su coeficiente β_i), y el modelo ajusta estos valores para minimizar el error en las predicciones. Es especialmente útil cuando la variable a predecir está influenciada por más de un parámetro. Al igual que en el caso anterior, se pueden usar métricas como el error cuadrático medio o el coeficiente de determinación (R^2) para evaluar su rendimiento.

Para que los resultados del modelo de regresión lineal sean válidos, deben cumplirse ciertos supuestos estadísticos:

- **Linealidad:** la relación entre las variables independientes y la variable dependiente es lineal.

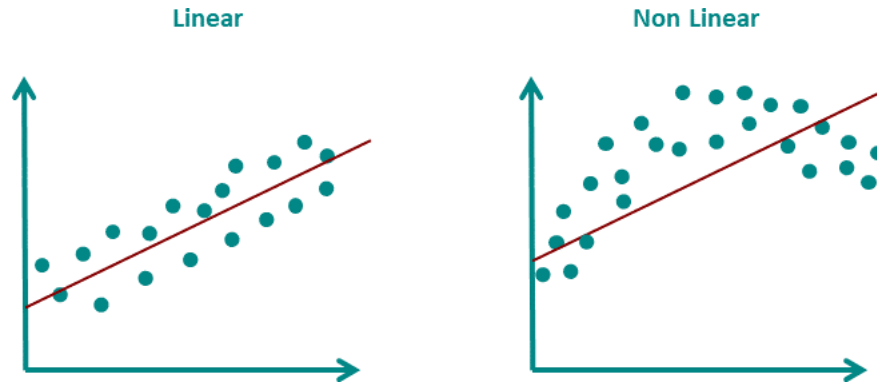


Figura 3: Lineal - No lineal

- **Independencia:** las observaciones son independientes entre sí.
- **Homoscedasticidad:** la varianza del término de error es constante para todos los valores de las variables independientes.

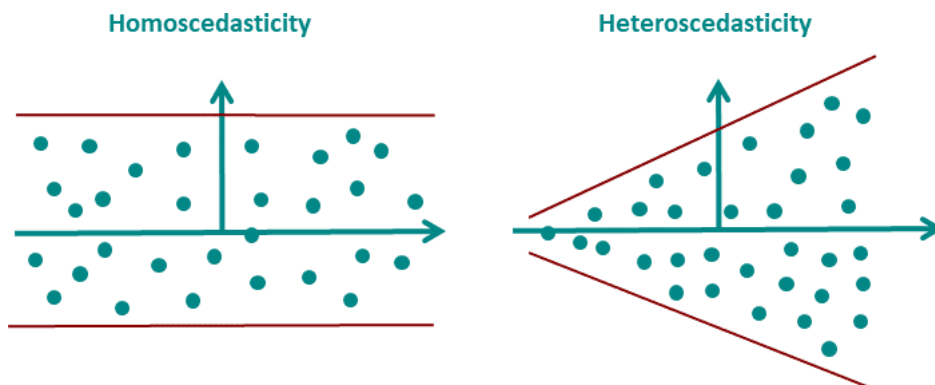


Figura 4: Homoscedasticidad

- **Normalidad de los residuos:** los errores del modelo se distribuyen normalmente.
- **Ausencia de multicolinealidad:** las variables independientes no deben estar altamente correlacionadas entre sí.

3.3. XGBoost

XGBoost es una biblioteca de *Machine Learning* de código abierto que utiliza árboles de decisión potenciados por gradiente, un algoritmo de *boosting* del aprendizaje supervisado que hace uso del descenso por gradiente. Es conocido por

su velocidad, eficacia y capacidad para escalar bien con grandes conjuntos de datos.

En este proyecto se ha comparado *XGBoost* con otros modelos de *Machine Learning* y *Deep Learning* para la previsión de inventario, utilizando métricas como el MAE y RMSE.

A continuación se explica como funciona *XGBoost* y por qué ha sido elegido como opción viable para la previsión de inventario.

XGBoost es un algoritmo de *boosting* basado en árboles de decisión que construye modelos de forma aditiva, optimizando una función objetivo compuesta por una función de pérdida y un término de regularización para evitar el sobreajuste. Utiliza derivadas de primer y segundo orden para mejorar la precisión y eficiencia en la búsqueda de divisiones óptimas en los árboles. [20] [21]

Incluye técnicas adicionales como *shrinkage* (reducción del impacto de cada árbol) y *submuestreo de columnas* (selección aleatoria de características) para mejorar la generalización. Ofrece algoritmos exactos y aproximados para encontrar las mejores divisiones, lo que permite adaptarse a distintos volúmenes y estructuras de datos.

XGBoost es especialmente adecuado para la previsión de inventario gracias a:

- **Alta escalabilidad:** puede trabajar con millones de registros incluso en entornos con recursos limitados, mediante computación fuera del núcleo y ejecución distribuida.
- **Manejo eficiente de datos dispersos:** ideal para bases de datos con muchos valores nulos o ceros.
- **Paralelización optimizada:** usa estructuras de datos que permiten procesamiento paralelo y rápido.
- **Uso eficiente de la memoria y caché:** minimiza cuellos de botella en el acceso a datos.
- **Compatibilidad con entornos distribuidos:** se integra con Spark, Hadoop, MPI, entre otros.

Estas características convierten a *XGBoost* en una herramienta robusta y eficiente para tareas de predicción en contextos reales, como la estimación de demanda o gestión de inventario en entornos empresariales.

3.4. Clustering

El *clustering* o agrupamiento nace de la necesidad de clasificar datos no en base a una variable objetivo sino en cuanto a su similaridad. Este método se encuadra bajo la rama de aprendizaje no supervisado, que busca ganar conocimiento a partir de datos no etiquetados. Ejemplos de uso para este algoritmo van desde las exploraciones iniciales de un conjunto de datos hasta el seguimiento de los procesos en curso.

Resulta útil pensar en el *clustering* como un intento de encontrar agrupaciones naturales en los datos para ver qué categorías pueden existir y qué define esas categorías. Pueden ayudar a encontrar relaciones subyacentes entre puntos de datos para ver qué características o rasgos se comparten entre las categorías.[22]

Existen varios tipos de *clustering* que se diferencian entre sí en base al objetivo que queramos conseguir y al conjunto de datos que estemos utilizando, teniendo en cuenta aspectos como: tamaño de los datos de entrada, dimensionalidad de los datos, rigidez de las categorías, etc. Nos vamos a centrar en tres tipos, pero especialmente en el primero:

- **Clustering basado en centroides:** Se basa en dividir un conjunto de datos en grupos similares en función de la distancia entre sus centroides. El centroide de cada clúster es la media o la mediana de todos los puntos del clúster, en función de los datos.

Una de las técnicas de *clustering* basado en centroides más utilizadas es el algoritmo de las **k-medias**. Este asume que el centro de cada clúster define el clúster utilizando una medida de distancia, normalmente la distancia euclidiana, al centroide.

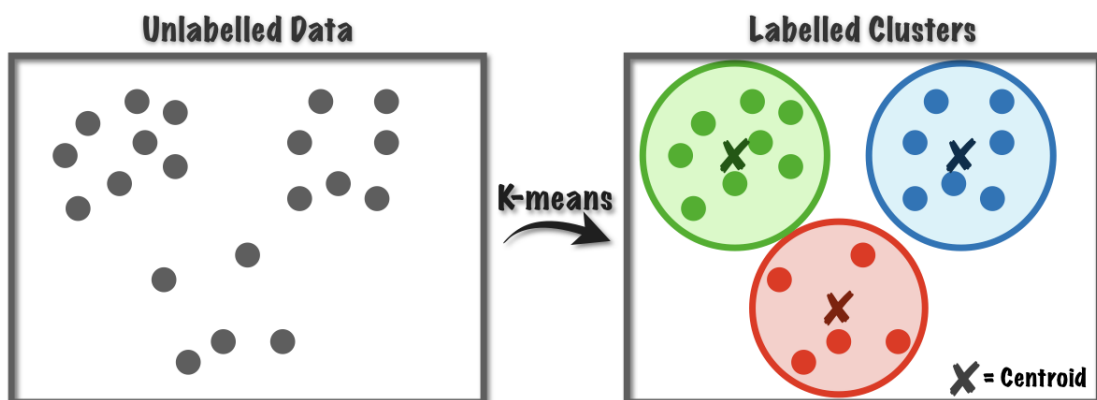


Figura 5: K-medias

K-medias utiliza un enfoque de *clustering* estricto, lo que significa que cada punto de datos se asigna a un clúster diferente y no hay ninguna probabilidad asociada a la pertenencia al clúster.

Para inicializar el *clustering*, se proporciona un número de clusters esperados, que representa la '**K**' en *K-medias*, y el algoritmo intenta encontrar clústers razonables a través de los datos que coincida con ese número. Este número de clústers no debe ser elegido al azar, existen distintas técnicas que permiten elegir con precisión el número óptimo de clústers para nuestros datos. Algunos de estos métodos son: **elbow method**, **gap statistics**, **the silhouette method**, etc.

Cada método tiene ventajas y desventajas únicas. Sin embargo, el *elbow method* o *método del codo* determina el número de clústers basándose en una representación gráfica que relaciona distintos valores del número de clusters con la suma de errores cuadráticos (SSE) correspondiente.

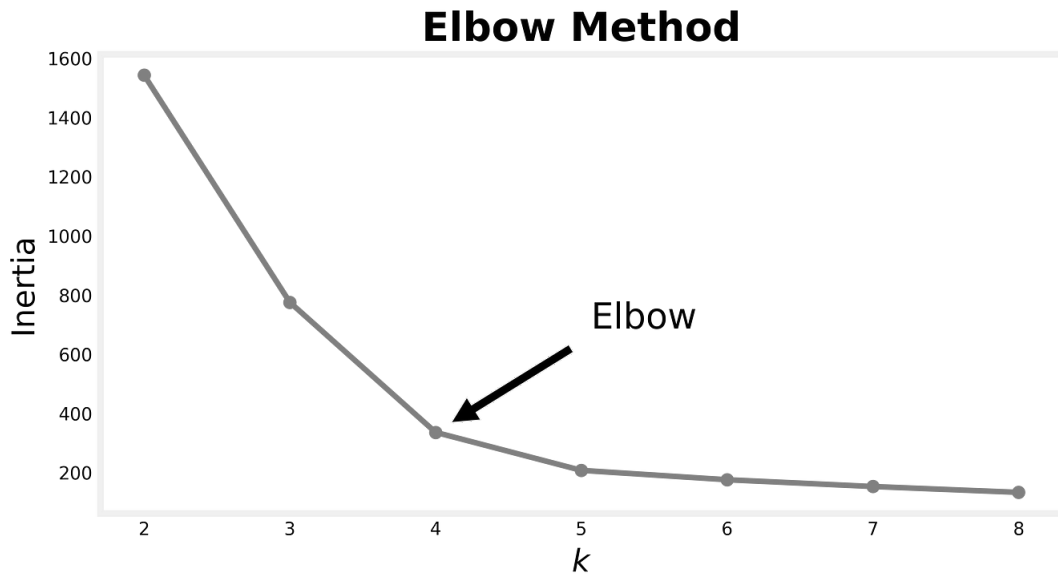


Figura 6: Método del Codo

El número ideal de clústers se identifica visualmente como el punto donde la curva forma un “codo” claramente marcado, lo que sugiere que añadir más agrupaciones no supone una reducción significativa del error. No obstante, una de las principales críticas de este método radica en su dependencia de la interpretación visual subjetiva, lo que puede conducir a una elección incorrecta del punto óptimo y, en consecuencia, afectar negativamente a la calidad del agrupamiento obtenido.[23]

- **Clustering jerárquico:** También llamado *clustering* basado en la conectividad, agrupa los puntos en función de la proximidad y la conectividad de sus atributos. Este método determina los clústeres en función de la proximidad de los puntos de datos entre sí en todas las dimensiones. La idea es que los puntos que están más cerca están más relacionados que los que están más lejos unos de otros.

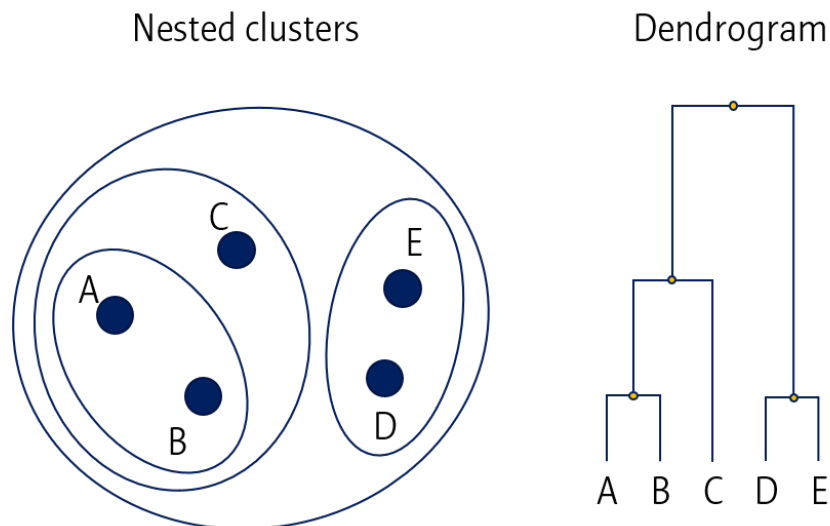


Figura 7: Clustering Jerárquico

A diferencia con el algoritmo de las *k*-medias, no es necesario especificar el número de clústeres. En su lugar, el algoritmo de clustering crea una red de grafos de los clústeres en cada nivel jerárquico. Esta red es jerárquica, lo que significa que cualquier nodo dado en ella solo tiene un nodo principal, pero puede tener varios nodos secundarios. Los clústeres jerárquicos se pueden representar gráficamente con un dendrograma para ayudar a resumir y organizar visualmente los clústeres detectados y la jerarquía que pueden contener.[24]

Existen dos enfoques para realizar el análisis jerárquico de clústeres:

El *clustering* aglomerativo sigue un enfoque ascendente, comenzando con cada punto como un clúster y fusionándolos sucesivamente según su proximidad hasta formar una jerarquía. Existen distintas estrategias de fusión (enlace simple, promedio, centroides, etc.), cada una con implicaciones en calidad y rendimiento. Entre sus desventajas destacan el encadenamiento y el alto coste computacional.

En cambio, el *clustering* divisivo emplea un enfoque descendente, dividiendo el conjunto completo en clústeres mediante métodos planos como *k-means*, y refinando divisiones según el error. Este método ofrece más flexibilidad estructural y puede ser más eficiente que el aglomerativo, especialmente con grandes conjuntos de datos.

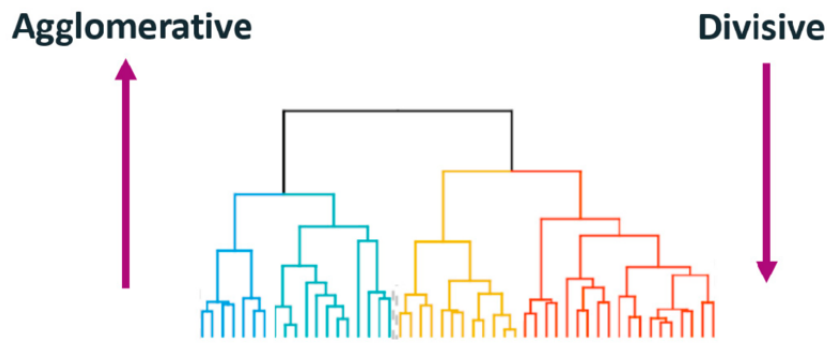


Figura 8: Aglomerativo - Divisivo

- **Clustering basado en densidad:** Su funcionamiento se basa en la detección de áreas donde se concentran puntos y donde están separados por áreas que están vacías o escasas. A diferencia de los enfoques basados en centroides, como el algoritmo de las k-medias, el clustering basado en la densidad puede detectar clústeres de una forma arbitraria. Puede ser muy útil cuando los clústeres no están definidos en torno a una ubicación o distribución específica.

DBSCAN utiliza este enfoque espacial basado en la densidad para definir clústeres con una densidad pasada por el usuario que se centra en un centroide espacial. El área inmediatamente alrededor del centroide se denomina vecindad y *DBSCAN* busca definir las vecindades de los clústeres que tienen la densidad pasada por el usuario. Para cada clúster, definirá tres tipos de puntos de datos:

- Puntos principales: un punto de datos es un punto central si la vecindad alrededor de ese punto de datos contiene al menos tantos puntos como el número mínimo de puntos especificado por el usuario.
- Puntos fronterizos: un punto de datos es un punto fronterizo si la vecindad alrededor de ese punto de datos contiene menos del número mínimo de puntos de datos pero la vecindad alrededor de ese punto contiene un punto central.
- Valor atípico: un punto de datos es un valor atípico si no es ni un punto central ni un punto de borde. Básicamente, esta es la 'otra' clase.

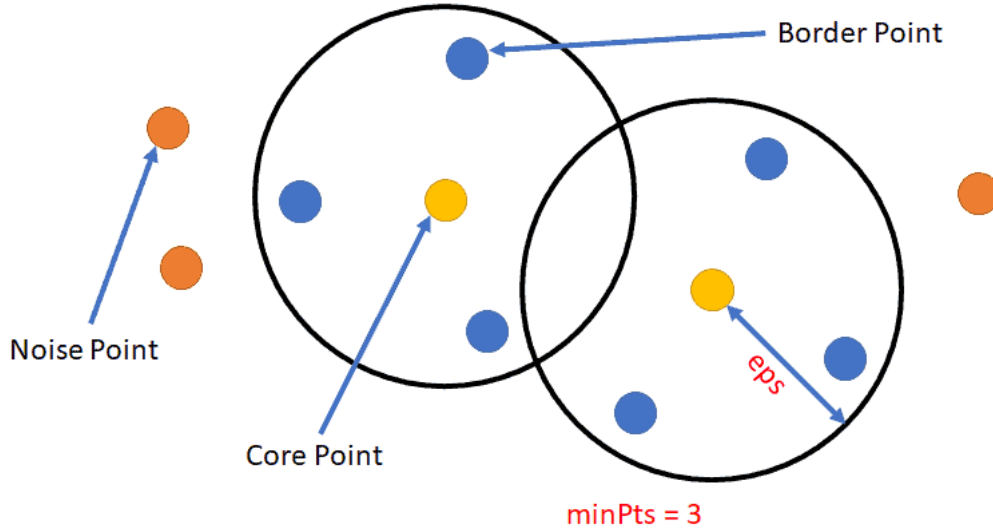


Figura 9: DBSCAN

4. Aprendizaje Profundo

El **aprendizaje profundo** (en inglés, *deep learning*) es una rama del *aprendizaje automático* que se basa en el uso de redes neuronales artificiales para modelar y comprender relaciones complejas dentro de grandes volúmenes de datos. Estas redes están inspiradas en el funcionamiento del cerebro humano y están compuestas de nodos o *neuronas* interconectadas y organizadas en capas. [25] [26]

4.1. Unidad de procesamiento: neurona

Una **neurona artificial** o *perceptrón* es la unidad básica de procesamiento en una red neuronal. Se inspira en el comportamiento de las neuronas biológicas y tiene como objetivo recibir una o varias entradas, procesarlas y generar una salida.

Cada entrada x_i se multiplica por un peso asociado w_i , y todas las entradas ponderadas se suman junto con un término adicional llamado *bias* o sesgo b . El resultado se pasa a través de una **función de activación** f , que introduce no linealidad al modelo, permitiendo que la red aprenda patrones complejos. Matemáticamente, el funcionamiento de una neurona artificial se puede expresar como:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

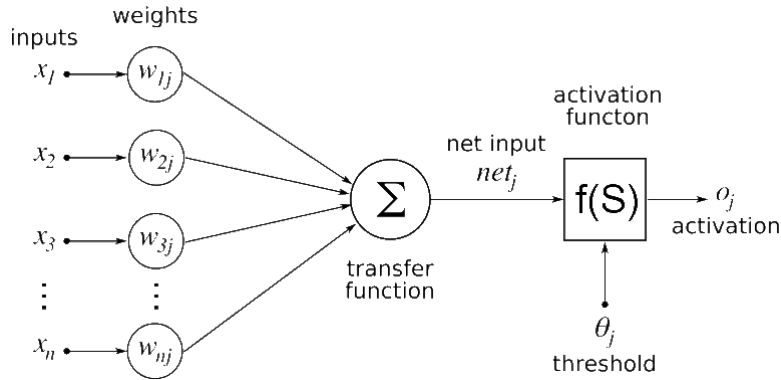
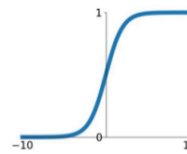


Figura 10: Neurona Artificial

La función de activación es elegida según el objetivo que se tenga a la hora de predecir, existen muchos tipos, pero las más conocidas son:

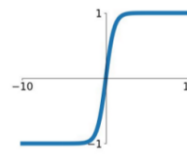
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$

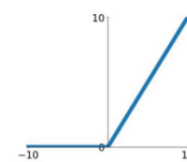


Figura 11: Funciones de activacion ANN's

Estas neuronas se organizan en capas, y su combinación da lugar a redes capaces de abordar tareas complejas como clasificación, regresión, reconocimiento de imágenes o predicción de series temporales.

4.2. Perceptrón Multicapa (MLP)

Para superar la limitación del **perceptrón simple** y modelar problemas no lineales (como XOR), se introducen **capas ocultas** entre la capa de entrada y la de salida. Esto da lugar a las **Redes Neuronales Feed-Forward** (*Feed-Forward Neural Networks*, FNNs), también conocidas como **Perceptrones Multicapa** (*Multi-Layer Perceptrons*, MLPs).

Una FNN está compuesta por capas de nodos (neuronas artificiales), distribuidas en una **capa de entrada**, una o varias **capas ocultas**, y una **capa de salida**.

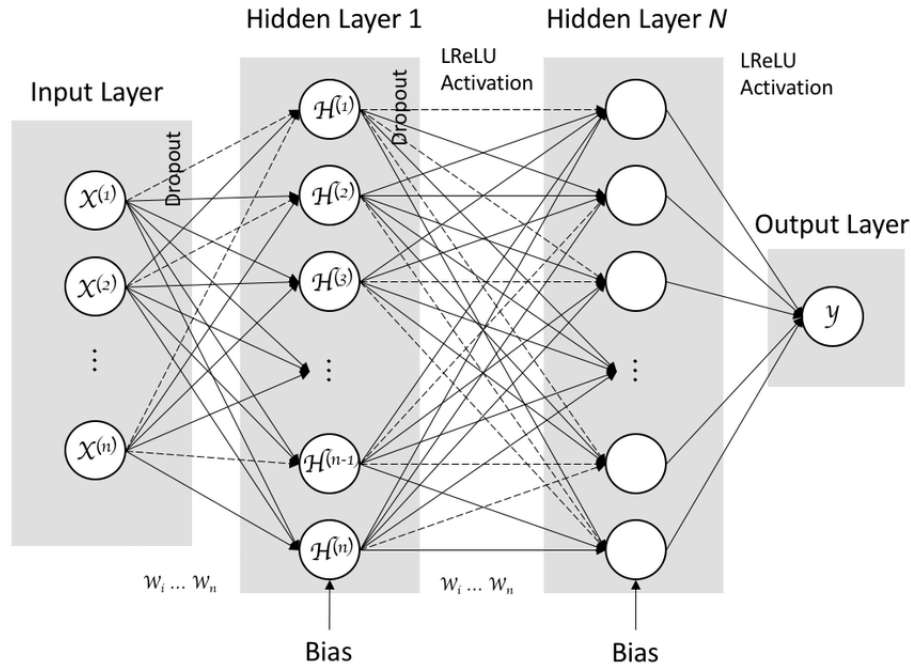


Figura 12: Ejemplo red MLP con dos capas ocultas.

Las FNNs con una sola capa oculta se consideran *shallow*, mientras que aquellas con al menos dos capas ocultas son clasificadas como *deep* (profundas).

En una FNN, los datos se mueven en una sola dirección (*hacia adelante*) a través de las capas, desde la entrada hasta la salida. La capa de entrada recibe los datos, las capas ocultas procesan los resultados de la capa anterior, y la capa de salida produce el resultado final.

Cada nodo está conectado a nodos de la capa siguiente y está asociado a un **peso** y un **sesgo** (*bias*). Los datos se transmiten si la salida supera ese umbral.

La **no linealidad** de las **funciones de activación** en las capas ocultas es esencial; sin ellas, la red solo podría modelar relaciones lineales.

Las FNNs, y específicamente las MLPs, son descritas como una variante sencilla y constituyen la **base de muchas arquitecturas más avanzadas** en el campo del aprendizaje profundo.

4.3. Red neuronal recurrente (RNN)

Las arquitecturas anteriores, como el **Perceptrón** y las **Redes Neuronales Feed-Forward** (FNN), asumen que los datos de entrada son independientes entre sí. Sin embargo, en muchos problemas reales como el análisis de texto, audio o series temporales, existe una clara **dependencia secuencial** entre los datos.

Las **Redes Neuronales Recurrentes** (*Recurrent Neural Networks*, RNNs) están diseñadas específicamente para trabajar con datos secuenciales, donde capturan la dinámica de las secuencias mediante **conexiones cíclicas** en la red.

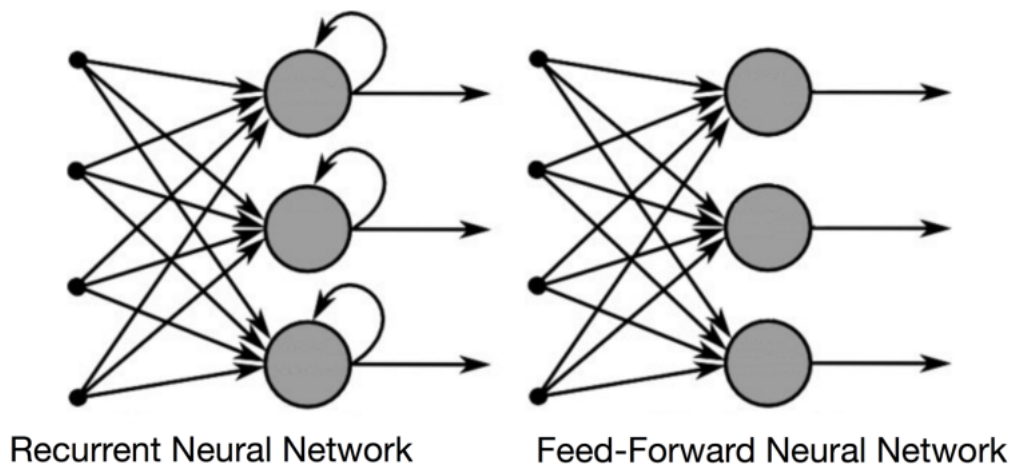


Figura 13: RNN - FNN.

A diferencia de las FNNs, las RNNs mantienen un **estado interno** que se actualiza en cada paso de tiempo, lo que permite que la red **recuerde** información de pasos anteriores.

El bloque computacional básico de una RNN es la **celda RNN**. En cada paso temporal t , esta celda recibe como entrada el dato actual $u^{(t)}$ y el **estado oculto** del paso anterior $h^{(t-1)}$, y a partir de ellos calcula un nuevo estado oculto $h^{(t)}$ y una salida $y^{(t)}$. Este comportamiento permite que la red adapte sus salidas en función del contexto acumulado en el tiempo.

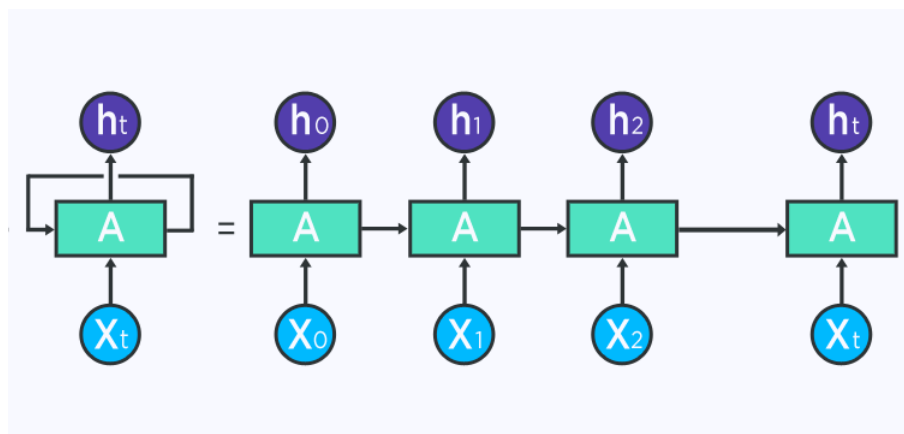


Figura 14: Esquema de una celda RNN en el tiempo.

Una característica clave de las RNNs es que los **pesos se comparten** a lo largo de todos los pasos de tiempo, lo que reduce significativamente el número de parámetros a entrenar y mejora la generalización.

A pesar de su capacidad para procesar secuencias, las RNNs estándar

enfrentan dificultades al aprender dependencias a largo plazo. Esto se debe a problemas durante la retropropagación del error a través del tiempo, lo que puede provocar que los *gradientes desaparezcan* o *exploten*. El llamado **problema del gradiente desvanecido** es particularmente común y ha motivado el desarrollo de arquitecturas más avanzadas como las LSTM.

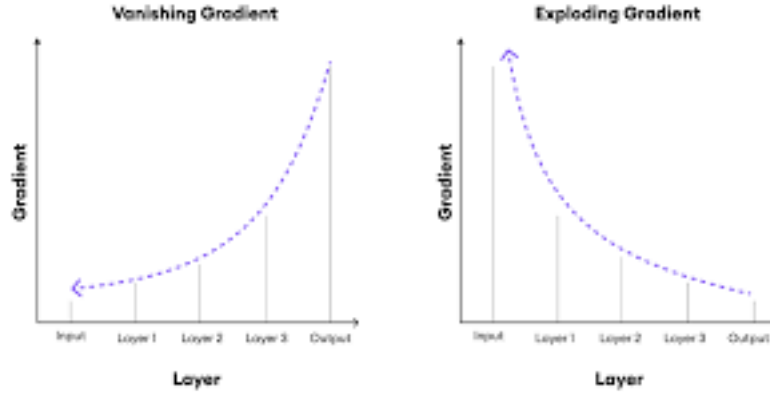


Figura 15: Problema del gradiente desvanecido.

4.4. Long Short Term Memory (LSTM)

Las Long Short Term Memory (LSTM) son una popular variante de las Redes Neuronales Recurrentes (RNNs). Nacieron con el objetivo de capturar mejor las dependencias a largo plazo y de evitar el problema del desvanecimiento del gradiente que aparecía en las RNNs estándar, especialmente con secuencias largas.

La arquitectura LSTM permite aplicar con mayor facilidad que las RNNs mecanismos para mitigar el problema del desvanecimiento del gradiente (gradient clipping). La idea central en la arquitectura de la celda LSTM es la adición de mecanismos de compuerta (gating mechanisms). [27] [28]

Estas compuertas actúan como una especie de 'memoria' que se va actualizando a lo largo del tiempo y permite que la información importante pase sin desaparecer. Para controlar qué se guarda, qué se olvida y qué se usa, se utilizan tres compuertas:

1. **Compuerta de olvido (forget gate) Γ_f :** Decide qué parte del estado de celda anterior $c^{(t-1)}$ se olvida.

$$\Gamma_f = \sigma \left(W_f \cdot [h^{(t-1)}; u^{(t)}] + b_f \right)$$

2. **Compuerta de entrada (input gate) Γ_i :** Decide qué nueva información se va a guardar en la celda.

$$\Gamma_i = \sigma \left(W_i \cdot [h^{(t-1)}; u^{(t)}] + b_i \right)$$

3. **Candidato a estado de celda** $\tilde{c}^{(t)}$: Genera nuevos valores candidatos para añadir al estado de celda.

$$\tilde{c}^{(t)} = \tanh \left(W_c \cdot [h^{(t-1)}; u^{(t)}] + b_c \right)$$

4. **Actualización del estado de celda** $c^{(t)}$:

$$c^{(t)} = \Gamma_f \odot c^{(t-1)} + \Gamma_i \odot \tilde{c}^{(t)}$$

donde $\odot$ representa la multiplicación elemento a elemento (Hadamard).

5. **Compuerta de salida (output gate)** Γ_o : Controla qué parte del estado de celda se convierte en salida.

$$\Gamma_o = \sigma \left(W_o \cdot [h^{(t-1)}; u^{(t)}] + b_o \right)$$

6. **Estado oculto (hidden state)** $h^{(t)}$:

$$h^{(t)} = \Gamma_o \odot \tanh(c^{(t)})$$

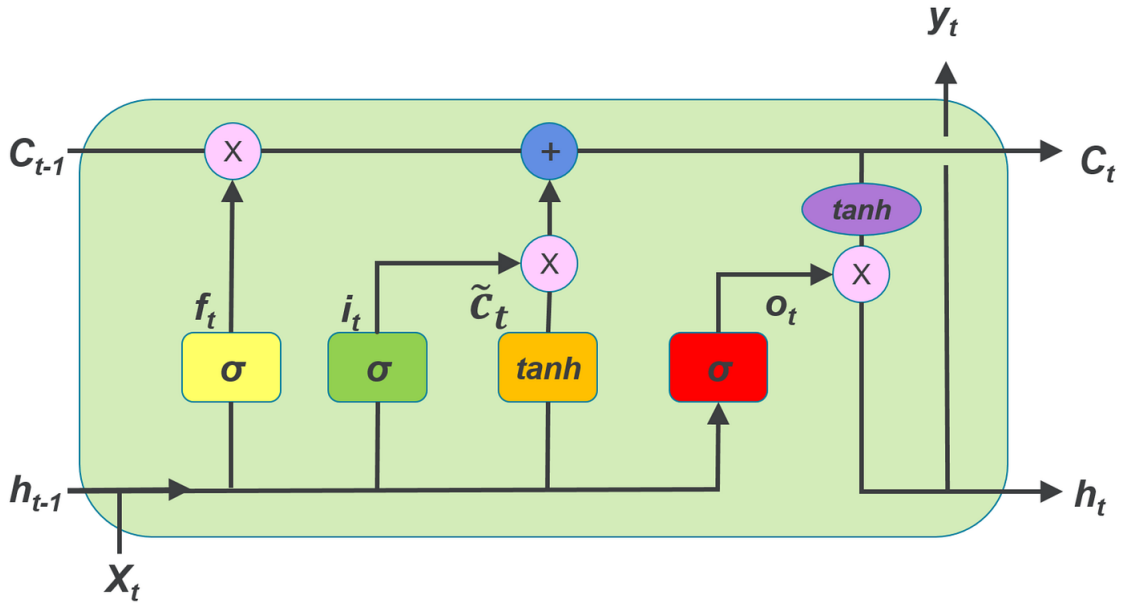


Figura 16: Celda LSTM.[1]

En resumen, las LSTMs mejoran a las RNNs estándar introduciendo un estado de celda y un sistema de compuertas que controlan el flujo de información, lo que les permite manejar dependencias a largo plazo de manera más efectiva y mitigar el problema del gradiente desvanecido.

Entrenamiento y evaluación de una LSTM

El entrenamiento de una red LSTM implica ajustar sus parámetros internos (pesos y sesgos) para minimizar el error entre las predicciones generadas y los valores reales de una secuencia temporal. Este proceso se realiza mediante **retropropagación a través del tiempo** (*Backpropagation Through Time*, BPTT), una extensión del algoritmo de retropropagación adaptada para redes recurrentes.

Dado un conjunto de entrenamiento compuesto por pares de secuencias (X, Y) , donde $X = \{x^{(1)}, x^{(2)}, \dots, x^{(T)}\}$ representa la secuencia de entrada y $Y = \{y^{(1)}, y^{(2)}, \dots, y^{(T)}\}$ la secuencia de salida esperada, la red LSTM genera una predicción $\hat{y}^{(t)}$ para cada paso temporal t .

La **función de pérdida** utilizada es típicamente el *error cuadrático medio* (MSE), definido como:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \left(y^{(t)} - \hat{y}^{(t)} \right)^2$$

Para minimizar esta pérdida, se utiliza un optimizador basado en descenso de gradiente, como *Adam* o *RMSProp*, que actualiza los parámetros en función de los gradientes calculados mediante BPTT.

Durante el entrenamiento, los gradientes se calculan hacia atrás en el tiempo a través de todos los pasos secuenciales, propagándose desde la salida final hasta los primeros pasos. Para evitar problemas de explosión o desaparición del gradiente, se pueden aplicar técnicas como **clipping de gradiente**, normalización o inicialización adecuada de pesos.

El proceso de entrenamiento se realiza en múltiples iteraciones (épocas), donde en cada una se recorre todo el conjunto de datos en pequeños lotes (*mini-batches*). Para evaluar el rendimiento del modelo y evitar el sobreajuste, se separa un subconjunto de datos para validación. Además, se puede utilizar el criterio de **early stopping**, que detiene el entrenamiento si la pérdida de validación deja de mejorar.

Una vez entrenada, la LSTM se evalúa utilizando métricas de regresión como:

- **MAE (Mean Absolute Error):**

$$MAE = \frac{1}{T} \sum_{t=1}^T \left| y^{(t)} - \hat{y}^{(t)} \right|$$

- **RMSE (Root Mean Squared Error):**

$$RMSE = \sqrt{\frac{1}{T} \sum_{t=1}^T \left(y^{(t)} - \hat{y}^{(t)} \right)^2}$$

Estas métricas permiten cuantificar la precisión del modelo y compararlo con otros enfoques. En el contexto de predicción de series temporales, como en la

estimación de demanda de productos, una LSTM bien entrenada puede capturar tanto patrones de corto como de largo plazo en los datos, siendo una herramienta potente en el aprendizaje profundo secuencial.

III. Planificación y costes

1. Documento de Requisitos

El documento de requisitos del sistema (DRS) tiene como objetivo recoger de forma estructurada los requisitos necesarios para el desarrollo del sistema propuesto en este Trabajo de Fin de Grado. La metodología seguida se basa en el informe técnico LSI-2000-10 [29] que proporciona una guía detallada para la elicitación, documentación y análisis de requisitos en proyectos de software.

1.1. Introducción

Siguiendo la guía anteriormente citada, antes de realizar el documento de requisitos del sistema, hay que tener en cuenta el siguiente esquema que resume perfectamente las tareas de elicitación de requisitos. Este esquema será el utilizado para elaborar este Trabajo de Fin de Grado adaptandolo a la situación y necesidades del proyecto.

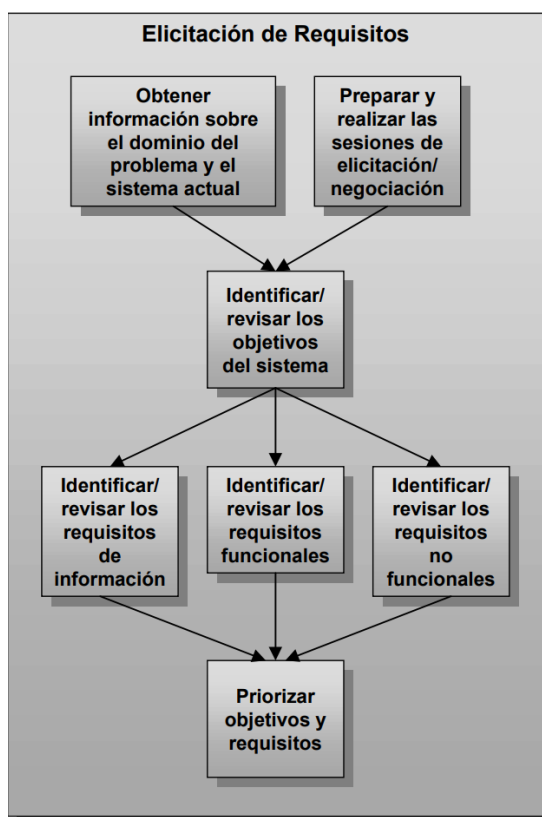


Figura 17: Tareas de elicitación de requisitos

La estructura del documento [29] definida a continuación es la siguiente:

- **Requisitos generales u objetivos del sistema (RG):** Son los objetivos que se pretenden alcanzar cuando el sistema software esté en explotación.
- **Requisitos de información (RI):** Describen la información que debe almacenar el sistema para ofrecer los servicios necesarios.
- **Requisitos funcionales (RF):** Servicios ofrecidos por el sistema.
- **Reglas de negocio (RN):** Restricciones establecidas en el sistema a desarrollar.
- **Requisitos no funcionales (RNF):** Condiciones impuestas al sistema a desarrollar relacionadas con aspectos de calidad que pueden influir en la arquitectura del sistema.
- **Matriz de trazabilidad de objetivos/requisitos:** Identifica las dependencias entre los diferentes tipos de requisitos y los objetivos del sistema.

En cuanto a la priorización de requisitos, se sigue el método *MoSCoW*, que define los siguientes niveles de prioridad:

- **Must (M):** Requisitos de máxima prioridad. No cumplirlos compromete el éxito del proyecto.
- **Should (S):** Requisitos de importancia alta. Son implementados luego de los *Must*.
- **Could (C):** Requisitos de importancia media o baja. Pueden ser implementados dependiendo del tiempo y presupuesto restante.
- **Won't (W):** Requisitos no previstos de implementar. No se ha añadido esta categoría en ningún requisito.

1.2. Requisitos Generales

En este apartado se resumen de forma global los aspectos más importantes del sistema que se va a construir. Los objetivos del sistema deben estar alineados con lo que se definió como alcance del proyecto y servir para que se puedan lograr esos propósitos.

RG-01	Uso de técnicas de ML
Versión	1.0
Descripción	El sistema debe integrar modelos de ML para análisis de datos.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Este requisito es clave en el proyecto, ya que representa la funcionalidad principal sobre la que se construye el sistema.

Figura 18: Cuadro de Requisitos Generales

RG-02	Aplicación web funcional
Versión	1.0
Descripción	Se deberá desarrollar una aplicación web funcional que permita al usuario interactuar con el sistema.
Prioridad	(M) Must
Estado	Hecho
Comentarios	La aplicación web sirve de interfaz principal del sistema.

RG-03	Visualización de métricas e información
Versión	1.0
Descripción	El sistema deberá mostrar al usuario información relevante sobre el inventario y las predicciones mediante gráficos y visualizaciones comprensibles.
Prioridad	(M) Must
Estado	Hecho
Comentarios	La visualización es clave para que el usuario entienda el estado del stock y las predicciones de forma rápida.

RG-04	Personalización del rango de fechas
Versión	1.0
Descripción	El sistema debería permitir al usuario seleccionar el rango de fechas para visualizar métricas históricas y consultar predicciones.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Esta funcionalidad mejora la utilidad de las visualizaciones, permitiendo analizar periodos concretos según las necesidades del usuario.

RG-05	Exportación de gráficas
Versión	1.0
Descripción	El sistema podría permitir exportar las gráficas en formato PNG o JPG.
Prioridad	(C) Could
Estado	Hecho
Comentarios	Esta funcionalidad sería útil para usuarios que deseen guardar los resultados fuera del sistema o compartirlos.

1.3. Requisitos de información

Los requisitos de información son especificaciones que definen qué datos debe almacenar, procesar o gestionar un sistema para cumplir con sus funcionalidades. Su objetivo es asegurar que el sistema disponga de la información necesaria para operar correctamente y ofrecer valor al usuario.

RI-01	Registro histórico de ventas
Versión	1.0
Descripción	El sistema debe almacenar y gestionar los datos históricos de ventas del negocio.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Es la base para las visualizaciones y modelos predictivos.

RI-02	Información de productos
Versión	1.0
Descripción	El sistema debe mantener información detallada de los productos (nombre, identificador, etc.).
Prioridad	(M) Must
Estado	Hecho
Comentarios	Necesario para análisis, visualización y agrupación.

RI-03	Métricas derivadas
Versión	1.0
Descripción	El sistema debe generar métricas a partir de los datos históricos (tendencias, totales, etc.).
Prioridad	(M) Must
Estado	Hecho
Comentarios	Se muestran en el dashboard.

RI-04	Predicciones de ventas
Versión	1.0
Descripción	El sistema debe generar y mostrar datos de predicción de ventas usando modelos ML.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Fundamental para la planificación de inventario.

RI-05	Agrupaciones de productos
Versión	1.0
Descripción	El sistema debe almacenar y mostrar los clústeres de productos generados mediante clustering.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Útil para comprender patrones de comportamiento comunes.

1.4. Requisitos funcionales

Los requisitos funcionales describen las acciones o comportamientos que el sistema debe ser capaz de realizar. Especifican las funciones concretas que debe ofrecer al usuario, como mostrar métricas, permitir la selección de productos para predicción, visualizar resultados o navegar entre módulos. Estos requisitos definen *qué* debe hacer el sistema desde el punto de vista del usuario o del proceso, y son esenciales para garantizar que cumple con los objetivos del proyecto.

RF-01	Panel de métricas
Versión	1.0
Descripción	El sistema debe mostrar un panel con gráficos y estadísticas de ventas.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Permite al usuario analizar el estado del negocio.

RF-02	Selección de producto para predicción
Versión	1.0
Descripción	El sistema debe permitir seleccionar un producto para predecir su demanda futura.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Entrada para los modelos de predicción.

RF-03	Visualización de predicción
Versión	1.0
Descripción	El sistema debe mostrar la predicción generada en un gráfico visual.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Facilita la comprensión de tendencias futuras

RF-04	Agrupación automática de productos
Versión	1.0
Descripción	El sistema debe agrupar productos según sus patrones de venta.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Complementa la toma de decisiones.

RF-05	Visualización de clústeres
Versión	1.0
Descripción	El sistema debe mostrar los productos agrupados y características de cada clúster.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Ayuda a interpretar resultados del clustering.

RF-06	Navegación entre módulos
Versión	1.0
Descripción	El sistema debe permitir cambiar entre secciones (métricas, predicción, clustering).
Prioridad	(M) Must
Estado	Hecho
Comentarios	Mejora la experiencia de usuario.

1.5. Reglas de negocio

Las reglas de negocio son condiciones, restricciones o directrices que definen o limitan el comportamiento del sistema desde el punto de vista operativo.

Establecen cómo deben funcionar ciertas funcionalidades bajo contextos específicos, asegurando coherencia, validez y alineación con los objetivos del negocio.

RN-01	Entrenamiento del modelo con datos históricos
Versión	1.0
Descripción	Los modelos de machine learning deben ser entrenados exclusivamente con los datos históricos de ventas almacenados en el sistema.
Prioridad	(M) Must
Estado	Hecho
Comentarios	

RN-02	Requisito de selección de producto para predicción
Versión	1.0
Descripción	El sistema no podrá generar una predicción si el usuario no ha seleccionado previamente un producto específico.
Prioridad	(M) Must
Estado	Hecho
Comentarios	

RN-03	Ajuste de métricas al rango de fechas
Versión	1.0
Descripción	Las métricas visualizadas deben corresponder al rango de fechas seleccionado por el usuario.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Mejora la utilidad de los análisis temporales sin ser crítica para el funcionamiento básico.

RN-04	Exportación de gráficas con datos existentes
Versión	1.0
Descripción	Solo podrán exportarse gráficas si estas han sido previamente generadas y están disponibles en pantalla.
Prioridad	(C) Could
Estado	Hecho
Comentarios	Esta regla asegura que no se intente exportar contenido inexistente, aunque no afecta funciones críticas del sistema.

1.6. Requisitos no funcionales

Los requisitos no funcionales definen las características de calidad que debe cumplir el sistema, sin centrarse en funciones específicas. Establecen criterios como la usabilidad, el rendimiento, la precisión de los modelos o la modularidad de la arquitectura.

RNF-01	Facilidad de uso
Versión	1.0
Descripción	El sistema debe tener una interfaz clara y entendible.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Crucial para usuarios sin formación técnica.

RNF-02	Rendimiento en tiempo real
Versión	1.0
Descripción	El sistema debe ofrecer respuestas rápidas, especialmente en predicciones.
Prioridad	(M) Must
Estado	Hecho
Comentarios	Afecta directamente a la usabilidad.

RNF-03	Modularidad del sistema
Versión	1.0
Descripción	El sistema debe tener componentes desacoplados para facilitar el mantenimiento.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Permite mejoras o cambios independientes.

RNF-04	Precisión de los modelos
Versión	1.0
Descripción	Los modelos de predicción deben alcanzar un nivel de precisión aceptable.
Prioridad	(S) Should
Estado	Hecho
Comentarios	Asegura confianza en las recomendaciones.

1.7. Matriz de trazabilidad

La Matriz de Trazabilidad es una herramienta útil que permite verificar que los requisitos generales del sistema están bien conectados con las reglas de negocio.

Gracias a esta matriz, se puede asegurar que todo lo que se ha definido tiene sentido dentro del funcionamiento del sistema y cumple con los objetivos planteados.

	RG-01	RG-02	RG-03	RG-04	RG-05
RI-01	X		X	X	
RI-02	X		X		
RI-03			X	X	
RI-04	X		X	X	
RI-05	X		X		
RF-01		X	X		X
RF-02	X	X			
RF-03	X	X	X		X
RF-04	X	X			
RF-05	X	X	X		X
RF-06		X			
RN-01	X				
RN-02	X	X			
RN-03			X	X	
RN-04		X	X		X
RNF-01		X	X		
RNF-02	X				

Figura 19: Matriz de trazabilidad

2. Alcance del proyecto

Esta sección contiene una descripción detallada del alcance del proyecto, es decir, el trabajo necesario para entregar un producto que cumpla con los objetivos generales. Además, se definen las exclusiones, restricciones y supuestos que delimitan y condicionan el alcance del proyecto.

2.1. Descripción del alcance

El alcance del proyecto abarca el desarrollo de un sistema de análisis y predicción de ventas para un entorno de negocio minorista, utilizando técnicas de

machine learning. El sistema incluirá una aplicación web funcional que permita al usuario visualizar métricas históricas, generar predicciones de demanda, analizar agrupaciones de productos y exportar resultados.

El trabajo a realizar comprende la recolección y tratamiento de datos históricos, el desarrollo e integración de modelos predictivos, la implementación del frontend con visualizaciones interactivas y la validación funcional del sistema. Todas las funcionalidades estarán orientadas a usuarios sin conocimientos técnicos, garantizando facilidad de uso y comprensión.

El alcance se limita a las funcionalidades especificadas en los requisitos del sistema, excluyendo aspectos como la autenticación de usuarios, la internacionalización o el reentrenamiento automático del modelo. También se establecen restricciones técnicas y metodológicas que guían el desarrollo del proyecto.

2.2. Exclusiones

Las exclusiones definen explícitamente qué funcionalidades o características quedan fuera del alcance del proyecto. Su objetivo es acotar el desarrollo, evitar desviaciones y centrar los esfuerzos en lo esencial.

EX-01	Autenticación de usuarios
Versión	1.0
Descripción	No se implementará un sistema de inicio de sesión o gestión de perfiles de usuario.
Estado	Hecho

EX-02	Internacionalización del sistema
Versión	1.0
Descripción	No se contempla la traducción de la interfaz ni la adaptación a diferentes monedas, formatos de fecha u otros aspectos regionales.
Estado	Hecho

EX-03	Actualización automática del modelo LSTM
Versión	1.0
Descripción	El modelo predictivo no se reentrena de forma continua con nuevos datos; su entrenamiento se realiza de forma estática durante el desarrollo.
Estado	Hecho

EX-04	Predicción personalizada por cliente
Versión	1.0
Descripción	No se contempla segmentar la demanda ni hacer recomendaciones según comportamiento de clientes individuales.
Estado	Hecho

2.3. Restricciones

Las restricciones del alcance son limitaciones que condicionan cómo debe desarrollarse el sistema y qué criterios debe cumplir para alinearse con los objetivos del proyecto.

RE-01	Cumplir los objetivos generales del proyecto
Versión	1.0
Descripción	El sistema debe implementar una solución funcional que responda a los objetivos definidos, centrados en el análisis y predicción de ventas mediante técnicas de <i>machine learning</i> .
Estado	Hecho

RE-02	Seguir un enfoque de desarrollo iterativo con control de versiones
Versión	1.0
Descripción	El proyecto debe desarrollarse siguiendo una metodología iterativa, con integración progresiva y control de versiones en GitHub
Estado	Hecho

RE-03	Usabilidad orientada a usuarios no técnicos
Versión	1.0
Descripción	La interfaz debe ser clara, intuitiva y comprensible por usuarios sin conocimientos técnicos, como el gerente de la frutería, evitando configuraciones avanzadas o técnicas.
Estado	Hecho

RE-04	Visualización interactiva de resultados
Versión	1.0
Descripción	Las métricas y predicciones deben representarse de forma visual e interactiva en el frontend, según lo definido en los requisitos funcionales del DRS.
Estado	Hecho

RE-05	Mantener la modularidad del sistema
Versión	1.0
Descripción	El sistema debe estar dividido en módulos claramente separados (frontend, backend, modelos ML) para facilitar el mantenimiento y la comprensión del código.
Estado	Hecho

2.4. Supuestos

Los supuestos son condiciones que se consideran verdaderas durante el desarrollo del proyecto, aunque no estén garantizadas. Sirven como base para la planificación y el diseño del sistema.

SU-01	Representatividad de los datos históricos
Versión	1.0
Descripción	Se asume que el conjunto de datos de ventas utilizado refleja fielmente el comportamiento real del negocio en el período analizado.
Estado	Hecho

SU-02	Se utiliza un Sistema de Control de Versiones (VCS) en la fase de Ejecución
Versión	1.0
Descripción	El proyecto debe utilizar un Sistema de Control de Versiones (VCS) como GitHub para llevar un seguimiento del código fuente, tareas pendientes y finalizadas.
Estado	Hecho

SU-03	Se valida el producto con el tutor en cada hito
Versión	1.0
Descripción	Después de cada hito, debe validarse el producto con el tutor del proyecto para realizar el seguimiento de las fases establecidas en el proyecto.
Estado	Hecho

SU-04	Usuario final sin conocimientos técnicos
Versión	1.0
Descripción	Se considera que el usuario objetivo del sistema (por ejemplo, el responsable del negocio) no tiene conocimientos avanzados en informática, por lo que todas las interacciones deben ser simples e intuitivas.
Estado	Hecho

3. Hitos del proyecto

HI-01	Entrega de documentos iniciales
Fecha de Vencimiento	20/03/2025
Versión	1.0
Estado	Hecho
Comentarios	Finalización del acta de constitución, alcance preliminar, objetivos y planificación general.

Figura 20: Hito 01

HI-02	Identificación y validación de requisitos
Fecha de Vencimiento	28/03/2025
Versión	1.0
Estado	Hecho
Comentarios	Documento de requisitos completado (funcionales, no funcionales, información, reglas de negocio, exclusiones, restricciones, supuestos).

Figura 21: Hito 02

HI-03	Finalización del diccionario EDT y cronograma
Fecha de Vencimiento	10/04/2025
Versión	1.0
Estado	Hecho
Comentarios	EDT detallado, actividades desglosadas y planificación temporal validada.

Figura 22: Hito 03

HI-04	Preparación del entorno de desarrollo
Fecha de Vencimiento	12/04/2025
Versión	1.0
Estado	Hecho
Comentarios	Instalación de herramientas, configuración del repositorio y pruebas iniciales del entorno.

Figura 23: Hito 04

HI-05	Desarrollo del modelo y procesamiento de datos
Fecha de Vencimiento	20/04/2025
Versión	1.0
Estado	Hecho
Comentarios	Datos preparados y modelo de ML entrenado y listo para su integración.

Figura 24: Hito 05

HI-06	Implementación del sistema (frontend + backend)
Fecha de Vencimiento	25/05/2025
Versión	1.0
Estado	Hecho
Comentarios	Sistema funcional con visualizaciones, predicciones y navegación entre módulos.

Figura 25: Hito 06

HI-07	Validación final del sistema con el tutor
Fecha de Vencimiento	30/05/2025
Versión	1.0
Estado	Hecho
Comentarios	Revisión completa del sistema y comprobación de cumplimiento de requisitos.

Figura 26: Hito 07

4. Estructura de Descomposición del Trabajo (EDT)

La EDT muestra de forma clara y ordenada los entregables del proyecto, agrupándolos por fases del PMBOK (Inicio, Planificación, Ejecución, Seguimiento y Control, Cierre). Cada bloque incluye actividades con sus tareas específicas, lo que asegura que todo esté alineado con los objetivos del proyecto. Esta estructura ayuda a gestionar mejor el trabajo, ya que permite tener una visión global, hacer seguimiento del avance y asignar bien los recursos.

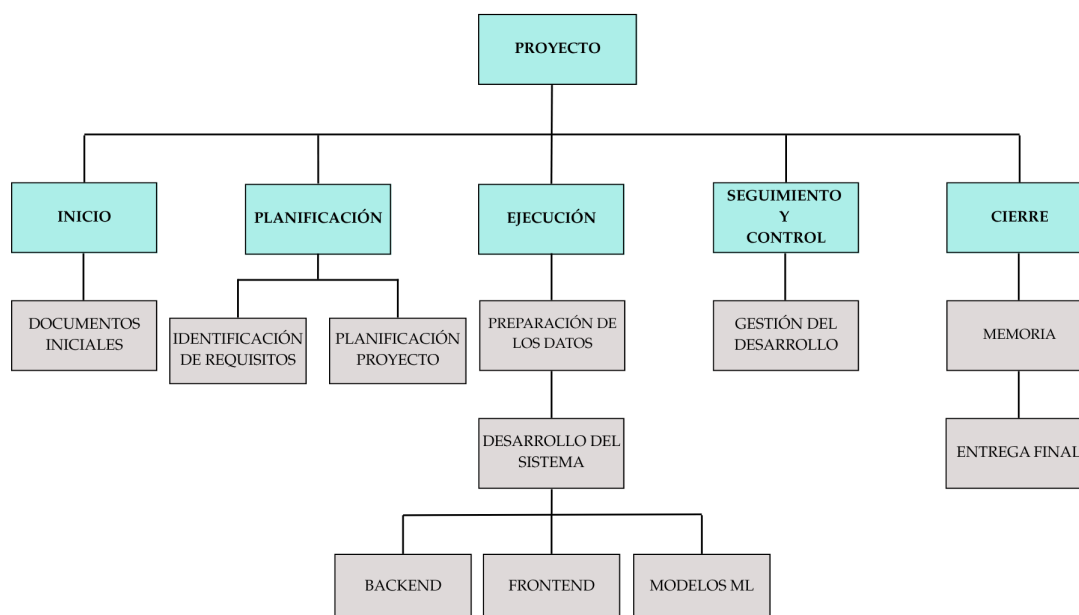


Figura 27: Estructura de Descomposición del Trabajo (EDT)

4.1. Diccionario EDT

Según la Guía PMBOK (R) en su versión 7, el diccionario de la EDT «es un documento que proporciona información detallada sobre los entregables, actividades y programación de cada uno de los componentes de la EDT».

CÓDIGO DE CUENTA	1.X	PAQUETE TRABAJO	Documentos iniciales
DESCRIPCIÓN	Redacción del acta de constitución del proyecto, definición del alcance preliminar, objetivos generales y planificación global inicial.		
FECHA INICIO	01/03/2025	FECHA FIN	20/03/2025
HITO		SUPUESTOS	RESTRICCIONES
HI-01		SU-04	RE-01

Figura 28: Paquete de trabajo 1.X: Documentos iniciales

CÓDIGO DE CUENTA	2.X	PAQUETE TRABAJO	Identificación de requisitos
DESCRIPCIÓN	Recogida, análisis y validación de requisitos funcionales, no funcionales, de información, generales, restricciones, supuestos y exclusiones.		
FECHA INICIO	21/03/2025	FECHA FIN	28/03/2025
HITO		SUPUESTOS	RESTRICCIONES
HI-02		SU-03	RE-01, RE-03

Figura 29: Paquete de trabajo 2.X: Identificación de requisitos

CÓDIGO DE CUENTA	3.X	PAQUETE TRABAJO	Planificación del proyecto
DESCRIPCIÓN	Definición de la EDT, cronograma detallado, diccionario de la EDT, estimación de tiempos y tareas y organización temporal del desarrollo.		
FECHA INICIO	29/03/2025	FECHA FIN	10/04/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-03	SU-03	RE-02	

Figura 30: Paquete de trabajo 3.X: Planificación del proyecto

CÓDIGO DE CUENTA	4.X	PAQUETE TRABAJO	Preparación de los datos
DESCRIPCIÓN	Carga, limpieza y transformación del dataset para su análisis, cálculo de métricas, validación de estructura, y generación de visualizaciones iniciales.		
FECHA INICIO	11/04/2025	FECHA FIN	19/04/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-05	SU-01, SU-02, SU-03	RE-01, RE-02, RE-05	

Figura 31: Paquete de trabajo 4.X: Preparación de los datos

CÓDIGO DE CUENTA	5.X	PAQUETE TRABAJO	Desarrollo del sistema
DESCRIPCIÓN	Desarrollo conjunto del sistema dividido en tres módulos principales: backend, frontend y modelos ML. Integra los datos con la interfaz y visualizaciones interactivas.		
FECHA INICIO	20/04/2025	FECHA FIN	25/05/2024
HITO	SUPUESTOS	RESTRICCIONES	
HI-06	SU-02, SU-04	RE-03, RE-04, RE-05	

Figura 32: Paquete de trabajo 5.X: Desarrollo del sistema

CÓDIGO DE CUENTA	6.X	PAQUETE TRABAJO	Backend
DESCRIPCIÓN	Implementación de la lógica del servidor con Flask, API REST y conexión con modelos predictivos y el sistema de visualización.		
FECHA INICIO	20/04/2025	FECHA FIN	10/05/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-06	SU-02	RE-05	

Figura 33: Paquete de trabajo 6.X: Backend

CÓDIGO DE CUENTA	7.X	PAQUETE TRABAJO	Frontend
DESCRIPCIÓN	Desarrollo de la interfaz de usuario en React, visualización de métricas y navegación entre módulos, con diseño accesible y centrado en el usuario.		
FECHA INICIO	20/04/2025	FECHA FIN	20/05/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-06	SU-04	RE-03	

Figura 34: Paquete de trabajo 7.X: Frontend

CÓDIGO DE CUENTA	8.X	PAQUETE TRABAJO	Modelos ML
DESCRIPCIÓN	Entrenamiento e integración de modelos de predicción de ventas (regresión lineal y LSTM) y clustering de productos. Evaluación de métricas y precisión.		
FECHA INICIO	11/04/2025	FECHA FIN	25/04/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-05	SU-01, SU-02	RE-01, RE-05	

Figura 35: Paquete de trabajo 8.X: Modelos ML

CÓDIGO DE CUENTA	9.X	PAQUETE TRABAJO	Gestión del desarrollo
DESCRIPCIÓN	Seguimiento de tareas en GitHub, reuniones con el tutor para validación de fases, revisión de avances respecto a cronograma y validación final del sistema.		
FECHA INICIO	01/04/2025	FECHA FIN	30/05/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-07	SU-03	RE-01	

Figura 36: Paquete de trabajo 9.X: Gestión del desarrollo

CÓDIGO DE CUENTA	10.X	PAQUETE TRABAJO	Memoria
DESCRIPCIÓN	Redacción de la memoria del proyecto, incluyendo justificación técnica, desarrollo, evaluación, resultados y lecciones aprendidas. Preparación del borrador.		
FECHA INICIO	01/06/2025	FECHA FIN	10/06/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-08	SU-03	RE-01	

Figura 37: Paquete de trabajo 10.X: Memoria

CÓDIGO DE CUENTA	11.X	PAQUETE TRABAJO	Entrega Final
DESCRIPCIÓN	Entrega oficial de la versión final de la memoria tras la revisión del tutor, cumpliendo con los requisitos formales de la ETSII y la convocatoria oficial.		
FECHA INICIO	11/06/2025	FECHA FIN	15/06/2025
HITO	SUPUESTOS	RESTRICCIONES	
HI-09	SU-03	RE-01	

Figura 38: Paquete de trabajo 11.X: Entrega Final

5. Actividades y estimaciones

5.1. Estimación de Recursos

Para calcular correctamente el presupuesto del proyecto, se han estimado los recursos necesarios, tanto humanos como físicos. La valoración se basa en tarifas de mercado según el “Informe de precios de perfiles informáticos” de la Junta de Andalucía y precios de licencias mensuales de las herramientas utilizadas.

Recursos Humanos

El trabajo ha sido realizado principalmente por un perfil técnico de tipo **Analista Programador Web (Junior)**, con la supervisión puntual de un **Director de Proyecto**. A continuación, se muestra la estimación de costes:

El desarrollo del trabajo ha sido llevado a cabo principalmente por un perfil técnico de tipo **Analista Programador Web (Junior)**, contando con la supervisión de un **Director de Proyecto**. Asimismo, se ha contado con la participación de un **ML Engineer**, responsable de la implementación de la parte de **Machine Learning**, y con la colaboración puntual de un **Analista de Requisitos**.

RECURSOS HUMANOS			
Perfil	Coste/hora (€)	Horas estimadas	Coste Total (€)
Analista de Requisitos	29,11	16	465,76
ML Engineer	30,33	149,5	4534,335
Analista Programador Web (Junior)	28,37	165,00	4681,05
Director de Proyecto	42,50	64,00	2720
Total estimado		394,50	12.401,15

Figura 39: Cuadro de recursos humanos

Recursos Físicos

Durante el proyecto se han empleado diversas herramientas y plataformas. El coste se ha calculado sobre una base de **3 meses de uso**, con precios mensuales aproximados:

RECURSOS FÍSICOS			
Recurso	Descripción	Coste Mensual (€)	Coste Total (€)
Ordenador personal	Uso estimado durante el proyecto	200,00	600,00
Onedrive	Almacenamiento en la nube	10,00	30,00
Github	Sistema de control de versiones	21,00	63,00
Visual Studio Code	Entorno de desarrollo	0,00	0,00
Jupyter Notebook	Herramienta análisis de código	0,00	0,00
GanttPro	Prueba gratuita	0,00	0,00
Overleaf	herramienta edición de documentos LATEX	19,00	57,00
Total estimado			750,00

Figura 40: Cuadro de recursos físicos

5.2. Actividades

Se elaboraron los documentos fundacionales del proyecto (acta, alcance, objetivos y planificación inicial), fundamentales para definir el marco, dirección y estrategia general antes del desarrollo técnico.

Paquete de Trabajo: 1.X – Documentos iniciales					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
1.1	Acta constitución del proyecto	Ordenador personal, Onedrive, overleaf	Director	4	170
1.2	Definición del alcance preliminar	Ordenador personal, Onedrive, overleaf	Analista Requisitos	4	116,44
1.3	Definición de los objetivos generales	Ordenador personal, Onedrive, overleaf	Director	2	85
1.4	Planificación global inicial del proyecto	Ordenador personal, OneDrive, Overleaf, Microsoft Project	Director	5	212,5
TOTAL				15	583,94

Figura 41: Cuadro actividades paquete de trabajo 1.X

Se identificaron, analizaron y validaron los requisitos necesarios para garantizar que el sistema cumpliera con las necesidades funcionales y de usuario. La validación con el tutor aseguró su alineación con los objetivos.

Paquete de Trabajo: 2.X – Identificación de requisitos					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
2.1	Recopilación de requisitos	Ordenador personal, Onedrive, overleaf	Analista Requisitos	5	145,55
2.2	Análisis y documentación de los requisitos recopilados	Ordenador personal, Onedrive, overleaf	Analista Requisitos	7	203,77
2.3	Validación de requisitos con el tutor del proyecto	Ordenador personal, Onedrive, overleaf	Director	3	127,5
TOTAL				15	476,82

Figura 42: Cuadro actividades paquete de trabajo 2.X

Se definió la EDT, se elaboró el cronograma y se estimaron tiempos y tareas. Estas actividades estructuran el desarrollo y permiten un control efectivo del tiempo y recursos.

Paquete de Trabajo: 3.X – Planificación del proyecto					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
3.1	Definición de la EDT	Ordenador personal, Microsoft Project	Director	3	127,5
3.2	Elaboración diccionario EDT	Ordenador personal, overleaf	Director	3	127,5
3.3	Estimación de tiempos y tareas del proyecto	Ordenador personal, Microsoft Project	Director	4	170
3.4	Creación del cronograma detallado	Ordenador personal, Microsoft Project	Director	5	212,5
3.5	Revisión y validación de la planificación con el tutor	Ordenador personal, Microsoft Project	Director	1	42,5
TOTAL				16	680

Figura 43: Cuadro actividades paquete de trabajo 3.X

Se realizaron tareas clave de carga, limpieza, transformación y análisis inicial del dataset, imprescindibles para asegurar la calidad de los datos antes del modelado y visualización.

Paquete de Trabajo: 4.X – Preparación de los datos					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
4.1	Carga del conjunto de datos (dataset)	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	4	121,32
4.2	Limpieza de datos para eliminar inconsistencias	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	8	242,64
4.3	Transformación y estructuración de los datos	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	6	181,98
4.4	Cálculo de métricas iniciales del dataset	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	4	121,32
4.5	Validación de la estructura de los datos	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	4	121,32
4.6	Generar visualizaciones de exploración inicial	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	4	121,32
TOTAL				30	909,9

Figura 44: Cuadro actividades paquete de trabajo 4.X

Se integraron los tres módulos del sistema (frontend, backend, ML) en una solución unificada y funcional, cumpliendo con los requisitos de visualización, predicción y navegación.

Paquete de Trabajo: 5.X – Desarrollo del sistema					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
5.1	Integración de los módulos (backend, frontend, ML) en un sistema unificado	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior), ML Engineer	60	3522
5.2	Pruebas integrales del sistema integrado	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	5	141,85
TOTAL				65	3663,85

Figura 45: Cuadro actividades paquete de trabajo 5.X

Se desarrolló la lógica del servidor y las APIs necesarias para conectar la interfaz con los modelos ML y los datos, garantizando la comunicación eficiente entre componentes del sistema.

Paquete de Trabajo: 5.1.X – Backend					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
5.1.1	Diseño de la arquitectura del backend (Flask)	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	5	141,85
5.1.2	Implementación de la API REST del servidor con Flask	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	10	283,7
5.1.3	Integración de los modelos predictivos en el backend	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	6	170,22
5.1.4	Conexión del backend con el sistema de visualización (frontend)	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	4	113,48
5.1.5	Pruebas y depuración del módulo backend	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	4	113,48
TOTAL				29	822,73

Figura 46: Cuadro actividades paquete de trabajo 5.1.X

Se diseñó e implementó una interfaz accesible y funcional en React, con visualizaciones interactivas y navegación clara entre módulos para facilitar su uso por usuarios no técnicos.

Paquete de Trabajo: 5.1.X – Frontend					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
5.1.6	Diseño de la interfaz de usuario (diseño UI/UX accesible)	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	6	170,22
5.1.7	Desarrollo del frontend en React (componentes y páginas)	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	12	340,44
5.1.8	Integración de la visualización de métricas en la interfaz	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	8	226,96
5.1.9	Implementación de la navegación entre módulos/páginas	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	2	56,74
5.1.10	Pruebas y corrección del módulo frontend	Ordenador personal, Visual Studio Code, GitHub	Analista Programador Web (Junior)	3	85,11
TOTAL				31	879,47

Figura 47: Cuadro actividades paquete de trabajo 5.1.X

Se entrenaron modelos de predicción (regresión y LSTM) y clustering, y se evaluó su rendimiento. Luego, se integraron con el sistema para proporcionar predicciones útiles al usuario.

Paquete de Trabajo: 5.1.X – Modelos ML					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
5.1.11	Preparación de datos para el entrenamiento de modelos	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	7,5	227,475
5.1.12	Entrenamiento de modelo de regresión lineal para predicción	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	3	90,99
5.1.13	Entrenamiento de modelo LSTM para predicción de ventas	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	9	272,97
5.1.14	Evaluación y comparación de modelos predictivos entrenados	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	15	454,95
5.1.15	Implementación de algoritmo de clustering de productos	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	9	272,97
5.1.16	Evaluación de resultados del clustering de productos	Ordenador personal, Jupyter Notebook, GitHub	ML Engineer	4	121,32
5.1.17	Integración de modelos entrenados en la aplicación	Ordenador personal, Visual Studio Code, GitHub	ML Engineer	12	363,96
TOTAL				60	1.804,64

Figura 48: Cuadro actividades paquete de trabajo 5.1.X

Se realizó el seguimiento del proyecto mediante GitHub y reuniones periódicas con el tutor, lo que permitió asegurar el cumplimiento de plazos y validar avances clave.

Paquete de Trabajo: 6.X – Gestión del desarrollo					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
6.1	Seguimiento y gestión de tareas del proyecto en GitHub (Kanban)	Ordenador personal, GitHub	Director	4	170
6.2	Reuniones periódicas de seguimiento con el tutor	Ordenador personal	Director	4	170
6.3	Monitoreo del cronograma y ajustes de planificación	Ordenador personal, Microsoft Project	Director	2	85
6.4	Validación final del sistema con el tutor del proyecto	Ordenador personal	Director	2	85
TOTAL				12	510

Figura 49: Cuadro actividades paquete de trabajo 6.X

Se documentó todo el trabajo técnico y metodológico realizado, con análisis de resultados y conclusiones, asegurando una justificación clara y profesional del proyecto.

Paquete de Trabajo: 7.X – Memoria					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
7.1	Documentación de la justificación técnica y metodología	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	8	226,96
7.2	Documentación del desarrollo e implementación del sistema	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	12	340,44
7.3	Documentación de la evaluación de resultados obtenidos	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	10	283,7
7.4	Redacción de conclusiones y lecciones aprendidas	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	4	113,48
7.5	Revisión y corrección del borrador de la memoria	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	4	113,48
7.6	Revisión del borrador con el Director de Proyecto (tutor)	Ordenador personal, OneDrive, Overleaf	Analista Programador Web (Junior)	2	56,74
TOTAL				40	1.134,80

Figura 50: Cuadro actividades paquete de trabajo 7.X

Se preparó y entregó la versión final de la memoria, incorporando correcciones del tutor y asegurando el cumplimiento de los requisitos formales de la ETSII.

Paquete de Trabajo: 8.X – Entrega final					
ID	Descripción	Recursos físicos	Responsable	Horas Trabajo	Coste actividad (€)
8.1	Incorporación de correcciones finales en la memoria	Ordenador personal, OneDrive, Overleaf	Director	10	425
8.2	Formateo y preparación de la versión final de la memoria	Ordenador personal, Overleaf	Director	6	255
8.3	Entrega oficial de la memoria final del proyecto	Ordenador personal	Director	4	170
8.4	Aprobación final del proyecto por el tutor	Ordenador personal	Director	2	85
TOTAL				22	935

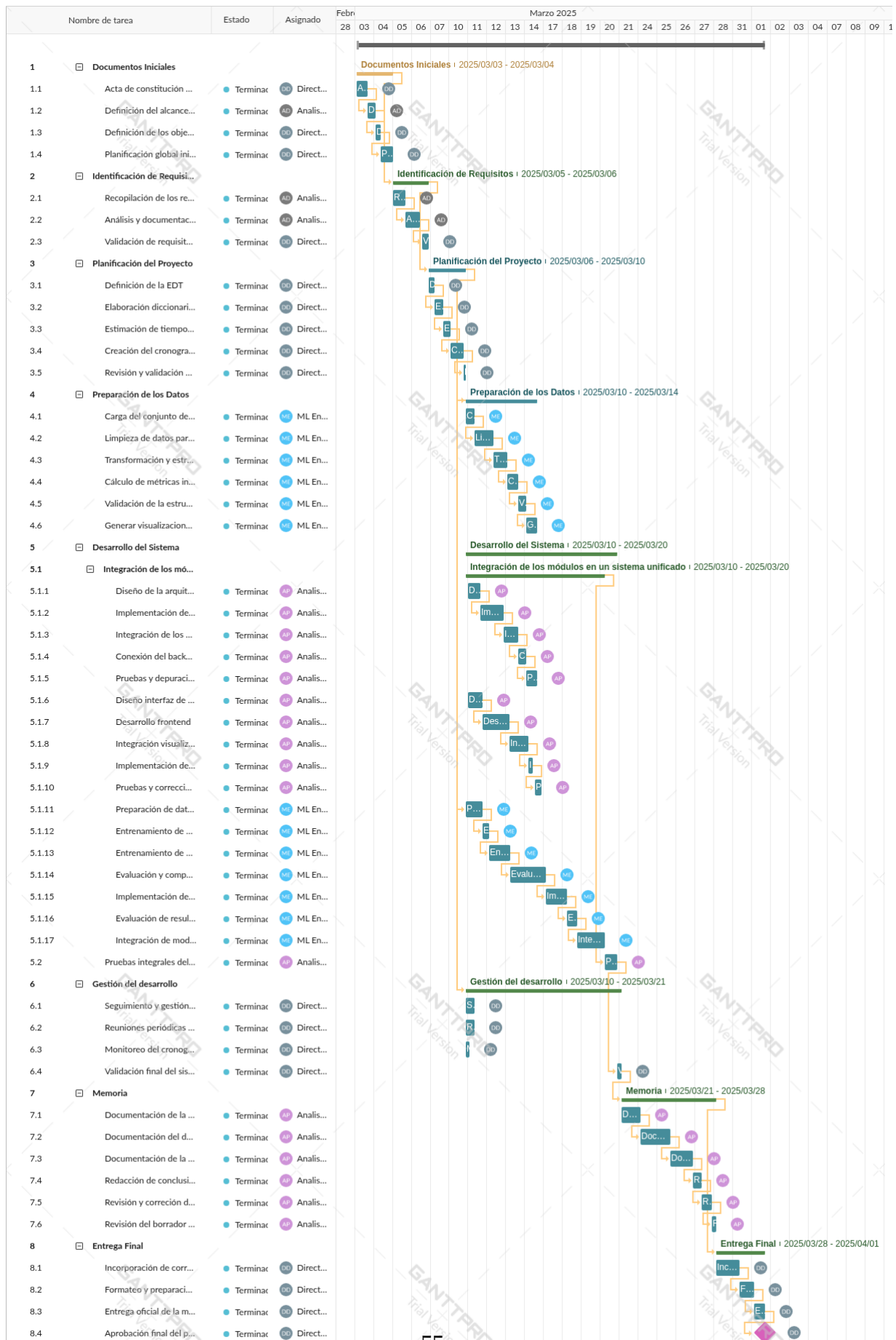
Figura 51: Cuadro actividades paquete de trabajo 8.X

6. Cronograma

El cronograma representa de forma estructurada el orden con el que se ha llevado a cabo el proyecto, reflejando la secuencia lógica de las actividades y su dependencia temporal. De esta manera, se facilita una visión integral de las fases del trabajo, que abarcan desde la elaboración de la documentación inicial y la identificación de requisitos, pasando por la planificación detallada y la preparación de los datos, hasta el desarrollo del sistema y la entrega final del TFG.

El diagrama de Gantt permite identificar con claridad los hitos principales, la duración prevista de cada tarea y las relaciones de precedencia entre ellas, constituyéndose como una herramienta fundamental para el seguimiento y control del proyecto. Gracias a esta representación, ha sido posible asegurar una adecuada

gestión del tiempo, optimizar la asignación de recursos y garantizar la coherencia en la ejecución de las diferentes etapas que conforman el trabajo.



7. Desviación de la planificación

Durante el desarrollo del proyecto han surgido algunas desviaciones respecto al cronograma inicial. Estas se deben sobre todo a dos factores: la falta de tiempo disponible y la necesidad de adquirir ciertos conocimientos que no estaban previstos al inicio.

El aprendizaje extra que se ha tenido que realizar (nuevas herramientas, técnicas de machine learning y aspectos de integración del sistema) ha requerido un esfuerzo adicional que no estaba contemplado en la planificación inicial. Sin embargo, este tiempo invertido en aprender no se suma oficialmente a las horas del proyecto, ya que se considera parte del proceso formativo personal.

A pesar de ello, las desviaciones no han impedido completar las tareas principales ni alcanzar los objetivos planteados, aunque sí han supuesto un reajuste en el ritmo de trabajo y en la distribución real de las horas.

8. Metodología de desarrollo

8.1. Comparación de metodologías de gestión de proyectos

Existen diversas metodologías para gestionar proyectos, cada una con sus ventajas, limitaciones y contextos ideales de aplicación. Elegir la más adecuada depende en gran medida del tipo de proyecto, del equipo de trabajo y del entorno en el que se desarrolla. Entre las más conocidas se encuentran el modelo en cascada, Scrum, Kanban y Extreme Programming (XP).

El **modelo en cascada** es uno de los enfoques más clásicos. Se basa en una secuencia lineal de fases: análisis, diseño, implementación, pruebas y mantenimiento. Cada etapa debe completarse antes de pasar a la siguiente, lo que implica poca o ninguna flexibilidad para hacer cambios una vez que se ha avanzado. Este enfoque puede funcionar en proyectos donde los requisitos están muy bien definidos desde el inicio, pero en entornos más cambiantes suele quedarse corto, ya que no se adapta bien a la incertidumbre.

En contraste, **Scrum** es una de las metodologías ágiles más populares. Propone dividir el trabajo en ciclos cortos llamados *sprints*, que suelen durar entre dos y cuatro semanas. Cada sprint tiene objetivos definidos y se revisa al final para evaluar el progreso. Scrum es ideal para equipos pequeños que pueden organizarse con roles claros (*Scrum Master*, *Product Owner*, equipo de desarrollo) y que necesitan trabajar con entregas frecuentes y mejoras continuas. Sin embargo, requiere una disciplina estricta en cuanto a reuniones, planificación y roles, lo cual puede ser un problema si el equipo es reducido o no tiene experiencia con este marco.

Por su parte, **Kanban** es también una metodología ágil, pero más flexible y menos estructurada que Scrum. No trabaja en iteraciones fijas, sino que promueve un flujo de trabajo continuo. El corazón de Kanban es el tablero, donde se

visualizan todas las tareas y su estado (por hacer, en proceso, hecho, etc.). A través de este sistema, se pueden identificar cuellos de botella, limitar el número de tareas en curso y mejorar progresivamente la eficiencia del equipo. Es especialmente útil en proyectos donde las prioridades cambian con frecuencia y se requiere adaptabilidad sin interrumpir el ritmo de trabajo.

Otra metodología ágil es **Extreme Programming (XP)**, centrada en la calidad del código y en la colaboración estrecha con el cliente. Introduce prácticas como la programación en pareja, desarrollo guiado por pruebas (*TDD*) y entregas muy frecuentes. Aunque puede mejorar mucho la calidad del producto, es exigente en cuanto a disciplina técnica y comunicación constante, por lo que no siempre es fácil de aplicar.

En definitiva, no existe una metodología perfecta. Cada una ofrece un enfoque distinto para organizar el trabajo. Mientras que el modelo en cascada puede funcionar bien en proyectos muy estables, los métodos ágiles como Scrum, Kanban o XP ofrecen herramientas más adecuadas para entornos dinámicos. En mi caso, he optado por Kanban porque necesito un sistema visual, simple y flexible que se adapte al ritmo del proyecto y permita ajustar prioridades sin rigideces innecesarias.

Existen muchas metodologías de desarrollo para proyectos software, las m Para organizar el trabajo se ha seguido una metodología de tipo **Kanban**, basada en una gestión flexible y visual del flujo de tareas. Esta metodología ha resultado especialmente útil al tratarse de un proyecto individual, ya que permite avanzar por objetivos concretos, adaptarse a los resultados obtenidos y mejorar cada componente del sistema de forma progresiva. [30]

8.2. Metodología utilizada

De esta manera, el desarrollo se ha planteado como un proceso iterativo, permitiendo avanzar por fases y realizar ajustes a medida que se completaban las distintas partes del sistema.

El proceso se ha dividido en varias etapas principales:

1. **Análisis inicial:** Se definieron los objetivos del sistema, como la predicción de ventas, la agrupación de productos y la integración de modelos en una aplicación web funcional.
2. **Preparación de los datos:** Se procesaron los datos disponibles, realizando tareas de limpieza, transformación y estructuración para su posterior uso en los modelos de aprendizaje automático.
3. **Entrenamiento de modelos:** Se entrenaron varios modelos adaptados a distintos tipos de datos. Por un lado, se realizó una comparación entre un modelo de regresión lineal y una red LSTM para series temporales. Por otro, se aplicaron algoritmos de clustering.
4. **Desarrollo del backend:** Se implementó el backend de la aplicación web

encargado de procesar los parametros de entrada del usuario en el frontend. A partir de estos datos, se generan las métricas y visualizaciones utilizando *Flask* y *CORS* los cuales permiten crear un entorno *API REST* para la continua actualización del frontend aportando así una interfaz interactiva. Además, estas métricas y visualizaciones se realizan con Python utilizando librerías como *pandas*, *numpy*, *bokeh* (para las visualizaciones), entre otras.

5. **Desarrollo del frontend:** Se implementó el frontend de la aplicación web encargado de la interfaz gráfica del producto. Para ello, se utilizó *React* ya que debido a su enfoque modular, permite tener un código estructurado, manejable y escalable.
6. **Pruebas y validación:** Se realizaron pruebas para comprobar el funcionamiento general del sistema y se evaluó el rendimiento de los distintos modelos utilizados, ajustando parámetros cuando fue necesario.

Este enfoque permitió construir un sistema completo de forma estructurada, adaptando y validando cada componente de manera progresiva.

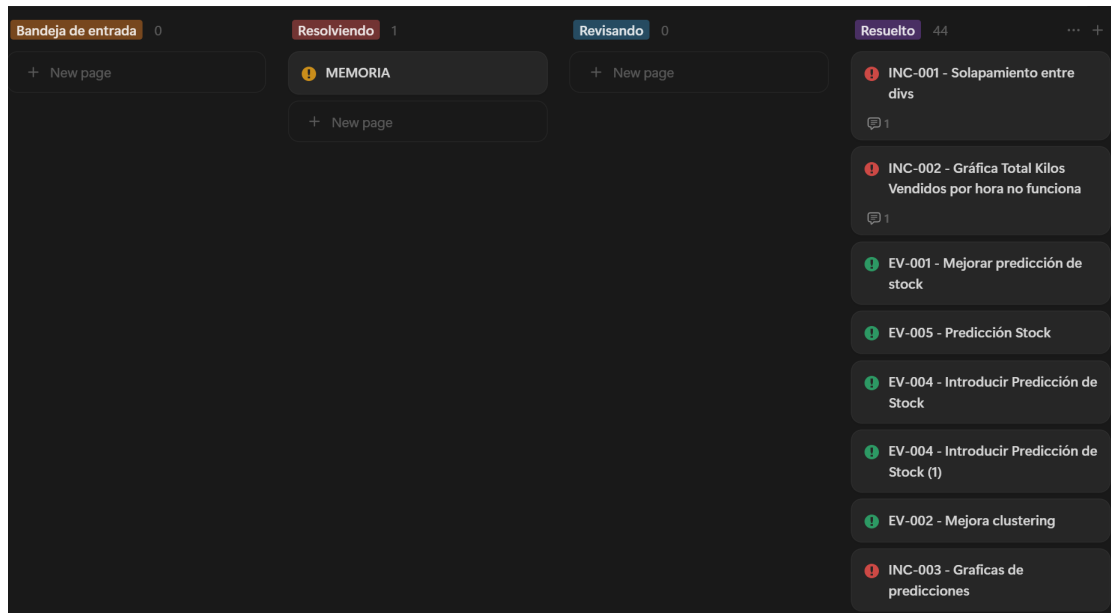


Figura 53: Panel de control Kanban

IV. Ejecución

1. Análisis exploratorio de los datos (EDA)

El análisis exploratorio de los datos es una de las tareas más relevantes dentro del campo de la Ciencia de Datos. Se utiliza con el objetivo de examinar e interpretar el conjunto de datos disponible, permitiendo a los técnicos comprender mejor su estructura y contenido.

Este proceso abarca desde la limpieza y preparación del dataset, que permite su posterior uso en herramientas de predicción, hasta la generación de visualizaciones que destacan características esenciales para interpretar el significado y comportamiento de los datos.

1.1. Búsqueda y elección del conjunto de datos

La elección del conjunto de datos ha sido una parte fundamental del proyecto, ya que de ella depende que las pruebas con los modelos tengan sentido y reflejen un caso realista. Desde el principio, la intención fue trabajar con datos representativos del día a día de un pequeño negocio de productos perecederos (por ejemplo, una frutería), donde se gestionan inventarios que pueden cambiar notablemente de una semana a otra.

Durante la búsqueda se estableció que el *dataset* debía cumplir ciertos requisitos básicos: contar con suficiente información temporal (histórico de ventas por fechas), ser lo más realista posible y permitir aplicar técnicas de predicción como modelos LSTM o regresión lineal. Además, era importante que los datos tuviesen poco ruido o, en su defecto, que pudieran prepararse fácilmente para el análisis (eliminando valores atípicos, completando faltantes, etc.).

Finalmente, el conjunto de datos seleccionado proviene de dos fuentes complementarias (`annex1.csv` y `dataset.csv`) que, combinadas, recogen el registro de ventas del negocio junto con información detallada de los productos. A continuación se procede a cargar ambos archivos y realizar una primera exploración de su contenido, verificando que se ajustan a los criterios definidos. [\[31\]](#)

1.2. Carga y visualización inicial de los datos

Lo primero que se hace es cargar los dos archivos `annex1.csv` y `dataset.csv`, ya que contienen la información necesaria para trabajar en el análisis. A continuación, se visualizan las primeras filas de cada uno para entender mejor

cómo están estructurados, qué tipo de variables contienen y si hay datos que puedan parecer extraños o faltantes.

```
1 import pandas as pd
2
3 df_annex1 = pd.read_csv('annex1.csv')
4 df_dataset = pd.read_csv('dataset.csv')
5
6 display(df_annex1.head())
7 display(df_dataset.head())
```

Extracto de código IV.1: Carga y visualización de los datos

En la Figura 54 se presenta el resultado de cargar y mostrar las primeras filas de cada conjunto de datos. Se aprecia la estructura de columnas de ambos *dataframes* y algunos valores de ejemplo. Por ejemplo, se pueden identificar campos comunes (como un código de producto) y valores numéricos o categóricos relevantes. Esta visualización inicial permite verificar que los datos se han importado correctamente y comenzar a detectar posibles valores atípicos o ausentes en una inspección preliminar.

	Item Code	Item Name	Category Code	Category Name
0	102900005115168	Niushou Shengcai	1011010101	Flower/Leaf Vegetables
1	102900005115199	Sichuan Red Cedar	1011010101	Flower/Leaf Vegetables
2	102900005115625	Local Xiaomao Cabbage	1011010101	Flower/Leaf Vegetables
3	102900005115748	White Caitai	1011010101	Flower/Leaf Vegetables
4	102900005115762	Amaranth	1011010101	Flower/Leaf Vegetables

	Date	Time	Item Code	Quantity Sold (kilo)	Unit Selling Price (RMB/kg)	Sale or Return	Discount (Yes/No)
0	2020-07-01	09:15:07.924	102900005117056	0.396	7.6	sale	No
1	2020-07-01	09:17:27.295	102900005115960	0.849	3.2	sale	No
2	2020-07-01	09:17:33.905	102900005117056	0.409	7.6	sale	No
3	2020-07-01	09:19:45.450	102900005115823	0.421	10.0	sale	No
4	2020-07-01	09:20:23.686	102900005115908	0.539	8.0	sale	No

Figura 54: Resultado carga y visualización de los datos

1.3. Preparación de los datos

En esta parte se preparan los datos para poder analizarlos bien. Lo primero es unir los dos datasets usando la columna *Item Code*, que es la que tienen en común. También se hace una conversión de precios de RMB a EUR usando una tasa fija, ya que trabajar con euros facilita mucho más interpretar los resultados y hacer análisis económicos después.

```
1 print("Columnas en df_annex1:", df_annex1.columns)
2 print("Columnas en df_dataset:", df_dataset.columns)
3
4 common_key = 'Item Code'
5
6 df_merged = pd.merge(df_dataset, df_annex1, on=common_key, how='left')
```

```

7
8 conversion_rate = 0.13
9 df_merged.insert(
10     df_merged.columns.get_loc('Unit Selling Price (RMB/kg)'),
11     'Unit Selling Price (EUR/kg)',
12     df_merged['Unit Selling Price (RMB/kg)] * 0.13
13 )
14 df_merged.drop(columns=['Unit Selling Price (RMB/kg)'], inplace=True)
15
16 display(df_merged.head())
17 display(df_merged.shape)

```

Extracto de código IV.2: Preparación de los datos

En el listado anterior, primero se imprimen las columnas de cada *DataFrame* original para confirmar la presencia de la clave común Item Code. Tras la fusión (*merge*) mediante esta clave, se agrega una nueva columna Unit Selling Price (EUR/kg) calculada a partir del precio en RMB, y se elimina la columna de precio original en RMB. Al visualizar las primeras filas del *DataFrame* resultante y sus dimensiones (Figura 55), se comprueba que la unión se ha realizado correctamente. La muestra de la cabecera indica que ahora cada fila de venta contiene también los datos propios del producto (categoría, tipo, etc.), y la nueva columna de precio en euros aparece en sustitución de la antigua. Por otra parte, el tamaño del *dataset* unificado (número de filas y columnas, mostrado también en la figura) revela que se mantiene el mismo número de registros que el *dataset* principal de ventas, lo cual indica que todos los productos de las ventas encontraron su correspondencia en el archivo anexo (no hubo ventas de productos desconocidos) y no se introdujeron duplicados en la combinación de datos.

Columnas en df_annex1: Index(['Item Code', 'Item Name', 'Category Code', 'Category Name'], dtype='object')

Columnas en df_dataset: Index(['Date', 'Time', 'Item Code', 'Quantity Sold (kilo)', 'Unit Selling Price (EUR/kg)', 'Sale or Return', 'Discount (Yes/No)', 'Item Name', 'Category Code', 'Category Name'], dtype='object')

	Date	Time	Item Code	Quantity Sold (kilo)	Unit Selling Price (EUR/kg)	Sale or Return	Discount (Yes/No)	Item Name	Category Code	Category Name
0	2020-07-01	09:15:07.924	102900005117056	0.396	0.988	sale	No	Paopaojiao (Jingpin)	1011010504	Capsicum
1	2020-07-01	09:17:27.295	102900005115960	0.849	0.416	sale	No	Chinese Cabbage	1011010101	Flower/Leaf Vegetables
2	2020-07-01	09:17:33.905	102900005117056	0.409	0.988	sale	No	Paopaojiao (Jingpin)	1011010504	Capsicum
3	2020-07-01	09:19:45.450	102900005115823	0.421	1.300	sale	No	Shanghaiqing	1011010101	Flower/Leaf Vegetables
4	2020-07-01	09:20:23.686	102900005115908	0.539	1.040	sale	No	Caixin	1011010101	Flower/Leaf Vegetables

(878503, 10)

Figura 55: Resultado Preparación de los datos

1.4. Exploración inicial del dataset unificado

Una vez que se tiene el *DataFrame* ya unificado y limpio, se empieza a explorar cómo son los datos en general. Se identifica cuántas filas y columnas hay, qué tipo de variables contiene y si hay valores nulos o duplicados. Todo esto es importante

para saber si los datos están en buen estado o si hay cosas que pueden afectar al análisis más adelante.

```
1 print("Dimensin del dataframe unido:")
2 print(df_merged.shape)
3
4 print("\nTipos de datos en el dataframe unido:")
5 df_merged.info()
6
7 print("\nEstadsticas descriptivas para columnas numricas:")
8 display(df_merged.describe())
9
10 print("\nNmero de valores nulos por columna:")
11 print(df_merged.isnull().sum())
12
13 print("\nNmero de filas duplicadas:")
14 print(df_merged.duplicated().sum())
```

Extracto de código IV.3: Exploración del DataFrame unido

La salida de estos comandos, resumida en la Figura 56, proporciona información valiosa sobre el *dataset* unificado. En primer lugar, el tamaño indicado por `shape` confirma las dimensiones del conjunto de datos tras la integración. A continuación, el informe de `info()` detalla el tipo de dato de cada columna y el número de entradas no nulas, lo que permite verificar que todas las columnas están pobladas con tipos apropiados (por ejemplo, numéricas, cadenas, fechas) y detectar si alguna columna contiene valores nulos de manera significativa.

Las estadísticas descriptivas muestran, para las variables numéricas, valores como media, desviación estándar, máximos y mínimos, dando una idea de la distribución de estos campos (por ejemplo, el rango de cantidades vendidas o de precios). Por último, el conteo de valores nulos por columna y el número de duplicados indican que el *dataset* no presenta problemas graves de calidad: la mayoría de las columnas no tienen valores faltantes (o sólo una proporción muy reducida) y no existen filas duplicadas (o su número es despreciable). Estos resultados sugieren que no será necesaria una limpieza exhaustiva antes de continuar con el análisis, dado que los datos están en buenas condiciones generales.

```

Dimensión del dataframe unido:
(878503, 10)

Tipos de datos en el dataframe unido:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 878503 entries, 0 to 878502
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Date                                878503 non-null  object
1   Time                                878503 non-null  object
2   Item Code                           878503 non-null  int64
3   Quantity Sold (kilo)                878503 non-null  float64
4   Unit Selling Price (EUR/kg)          878503 non-null  float64
5   Sale or Return                      878503 non-null  object
6   Discount (Yes/No)                   878503 non-null  object
7   Item Name                           878503 non-null  object
8   Category Code                       878503 non-null  int64
9   Category Name                       878503 non-null  object
dtypes: float64(2), int64(2), object(6)
memory usage: 67.0+ MB
Estadísticas descriptivas para columnas numéricas:

```

	Item Code	Quantity Sold (kilo)	Unit Selling Price (EUR/kg)	Category Code
count	8.785030e+05	878503.000000	878503.000000	8.785030e+05
mean	1.030313e+14	0.536112	1.159229	1.011010e+09
std	7.174368e+11	0.400087	0.820464	2.582929e+02
min	1.029000e+14	-9.082000	0.013000	1.011010e+09
25%	1.029000e+14	0.290000	0.637000	1.011010e+09
50%	1.029000e+14	0.435000	1.027000	1.011010e+09
75%	1.029000e+14	0.742000	1.404000	1.011011e+09
max	1.069740e+14	160.000000	15.587000	1.011011e+09

```

Número de valores nulos por columna:
Date                                0
Time                                0
Item Code                           0
Quantity Sold (kilo)                0
Unit Selling Price (EUR/kg)          0
Sale or Return                      0
Discount (Yes/No)                   0
Item Name                           0
Category Code                       0
Category Name                       0
dtype: int64

```

```

Número de filas duplicadas:
0

```

Figura 56: Resultado exploración del DataFrame unido

1.5. Análisis de distribución y relaciones entre variables

En esta parte se hace un análisis más detallado del dataset. Se revisan las variables categóricas para ver, por ejemplo, cuántos productos tienen descuento, cuántos son devueltos o a qué categoría pertenecen. También se analizan variables

numéricas como la cantidad vendida o el precio por kilo, lo que ayuda a detectar valores extremos o patrones curiosos.

Luego se calcula la correlación entre la cantidad vendida y el precio, para ver si existe alguna relación entre ambas (por ejemplo, si al bajar el precio se vende más). Además, se añaden columnas nuevas como los ingresos generados por cada venta, y se agrupan los datos por fecha y por categoría para ver tendencias a lo largo del tiempo.

```
1 print("Frecuencia de valores en 'Category Name':")
2 display(df_merged['Category Name'].value_counts())
3
4 print("\nFrecuencia de valores en 'Sale or Return':")
5 display(df_merged['Sale or Return'].value_counts())
6
7 print("\nFrecuencia de valores en 'Discount (Yes/No)':")
8 display(df_merged['Discount (Yes/No)'].value_counts())
9
10 print("\nEstadísticas resumen de 'Quantity Sold (kilo)':")
11 display(df_merged['Quantity Sold (kilo)'].describe())
12
13 print("\nEstadísticas resumen de 'Unit Selling Price (EUR/kg)':")
14 display(df_merged['Unit Selling Price (EUR/kg)'].describe())
```

Extracto de código IV.4: Análisis de variables categóricas y numéricas

Los resultados de las operaciones anteriores se resumen en la Figura 57. En cuanto a las variables categóricas, se observa que el conjunto de datos abarca varias categorías de producto, aunque unas pocas concentran la mayoría de los registros (es decir, la distribución de *Category Name* está sesgada hacia las categorías más frecuentes). Además, la mayoría de los registros corresponden a ventas normales: el valor *Sale* aparece mucho más frecuentemente que *Return*, lo cual indica que el volumen de devoluciones es relativamente bajo en comparación con las ventas efectivas. De forma similar, predomina el valor *No* en la columna de descuento, evidenciando que la mayor parte de las transacciones no incluyeron un descuento.

Por otro lado, las estadísticas descriptivas de las variables numéricas revelan información sobre su distribución: en la *Quantity Sold (kilo)* el valor máximo de kilos vendidos en una transacción es considerablemente superior a la media, lo que sugiere la existencia de algunos eventos de venta extraordinariamente grandes (valores atípicos) en comparación con el volumen típico de venta. En contraste, la mayor parte de las ventas parecen ser de volumen moderado (dada la diferencia entre cuartiles y el rango intercuartil observado). Para el *Unit Selling Price (EUR/kg)*, los estadígrafos resumen indican que la mayoría de los precios unitarios se sitúan en un rango intermedio, aunque hay presencia de precios extremos (muy bajos o muy altos) en menor proporción, que podrían corresponder a productos inusuales o promociones especiales.

En conjunto, este análisis preliminar de frecuencias y estadísticas proporciona un panorama general: las distribuciones de variables categóricas están dominadas por unos pocos valores frecuentes, y las variables numéricas presentan asimetrías que podrían ser relevantes a la hora de modelar (por ejemplo, colas largas en la

distribución de cantidades vendidas).

Frecuencia de valores en 'Category Name':

Category Name	
Flower/Leaf Vegetables	331968
Capsicum	207996
Edible Mushroom	148424
Cabbage	86570
Aquatic Tuberous Vegetables	58647
Solanum	44898

Name: count, dtype: int64

Frecuencia de valores en 'Sale or Return':

Sale or Return	
sale	878042
return	461

Name: count, dtype: int64

Frecuencia de valores en 'Discount (Yes/No)':

Discount (Yes/No)	
No	831137
Yes	47366

Name: count, dtype: int64

Figura 57: Resultado análisis de variables categóricas y numéricas

Estadísticas resumen de 'Quantity Sold (kilo)':

```
count      878503.000000
mean        0.536112
std         0.400087
min         -9.082000
25%         0.290000
50%         0.435000
75%         0.742000
max         160.000000
Name: Quantity Sold (kilo), dtype: float64
```

Estadísticas resumen de 'Unit Selling Price (EUR/kg)':

```
count      878503.000000
mean        1.159229
std         0.820464
min         0.013000
25%         0.637000
50%         1.027000
75%         1.404000
max         15.587000
Name: Unit Selling Price (EUR/kg), dtype: float64
```

```
1 correlation = df_merged['Quantity Sold (kilo)'].corr(df_merged['Unit Selling Price (EUR/
2 print(f"\nCorrelacin entre 'Quantity Sold (kilo)' y 'Unit Selling Price (EUR/kg)': {
3     correlation}")
4 df_merged['Date'] = pd.to_datetime(df_merged['Date'])
5 df_merged['Sales Revenue (EUR)'] = df_merged['Quantity Sold (kilo)'] * df_merged['Unit
6     Selling Price (EUR/kg)']
7 daily_trends = df_merged.groupby('Date').agg(
8     total_quantity_sold=('Quantity Sold (kilo)', 'sum'),
9     total_sales_revenue=('Sales Revenue (EUR)', 'sum')
10 ).reset_index()
11
12 print("\nTendencias diarias (primeras 5 filas):")
13 display(daily_trends.head())
14
15 category_sales = df_merged.groupby('Category Name').agg(
16     total_quantity_sold=('Quantity Sold (kilo)', 'sum'),
17     total_sales_revenue=('Sales Revenue (EUR)', 'sum')
18 ).reset_index()
19
20 print("\nPatrones de venta por 'Category Name':")
21 display(category_sales)
```

Extracto de código IV.5: Correlación y creación de métricas derivadas

La Figura 58 muestra los resultados obtenidos tras ejecutar estas operaciones. En primer lugar, el coeficiente de correlación calculado entre la cantidad vendida y el precio unitario resulta ser muy bajo (cercano a 0), tal como queda impreso en la

salida. Esto indica que no existe una relación lineal fuerte entre ambas variables en este conjunto de datos; es decir, el precio por kilo de un producto no parece determinar de forma evidente las variaciones en la cantidad vendida. Este hallazgo sugiere que factores adicionales (como la categoría del producto o si la venta tuvo descuento) podrían estar influyendo en las ventas más que el precio por sí solo, algo que deberá considerarse al construir modelos predictivos.

Por otra parte, la creación de la columna *Sales Revenue (EUR)* para cada registro permite calcular indicadores agregados de ventas en términos monetarios. Así, el DataFrame `daily_trends` resume las ventas diarias totales, acumulando la *cantidad* vendida y el *ingreso* generado por día; en la figura se muestran sus primeras filas a modo de ejemplo, donde cada fila corresponde a una fecha y resume toda la actividad de ventas de ese día.

De igual modo, el DataFrame `category_sales` contiene, para cada categoría de producto, la suma de las cantidades vendidas y de los ingresos totales asociados en el periodo considerado. La figura presenta la tabla completa de *category_sales*, lo que permite identificar qué categorías acumulan mayor volumen de ventas en kilos y en euros. Esta información agregada será especialmente útil de cara a las siguientes etapas del proyecto, ya que facilita la detección de patrones globales (por ejemplo, días con ventas excepcionalmente altas o categorías líderes en ingresos) que se exploran con más detalle mediante visualizaciones.

Correlación entre 'Quantity Sold (kilo)' y 'Unit Selling Price (EUR/kg)': -0.3741627643985289

Tendencias diarias (primeras 5 filas):

	Date	total_quantity_sold	total_sales_revenue
0	2020-07-01	404.346	451.361768
1	2020-07-02	393.678	449.350577
2	2020-07-03	385.018	440.020113
3	2020-07-04	483.299	571.332112
4	2020-07-05	516.917	581.304724

Patrones de venta por 'Category Name':

	Category Name	total_quantity_sold	total_sales_revenue
0	Aquatic Tuberous Vegetables	40581.353	45511.687091
1	Cabbage	41766.451	48847.721650
2	Capsicum	91588.629	98037.292171
3	Edible Mushroom	76086.725	80547.713246
4	Flower/Leaf Vegetables	198520.978	140279.074208
5	Solanum	22431.782	24846.153618

Figura 58: Resultado correlación y creación de métricas derivadas

Gracias a este análisis exploratorio se consigue una primera visión bastante útil sobre cómo se comportan las ventas. Esto permite detectar patrones que luego

pueden servir para tomar decisiones en fases más avanzadas del proyecto, como segmentar productos, ajustar precios o incluso predecir la demanda.

1.6. Visualización de las métricas

Distribución de la cantidad vendida

Para empezar con las visualizaciones, se representa un histograma que muestra cómo se distribuyen las cantidades vendidas (en kilos). Esto sirve para detectar si hay valores extremos o si la mayoría de las ventas se concentran en ciertos rangos.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 plt.figure(figsize=(10, 6))
5 sns.histplot(df_merged['Quantity Sold (kilo)'], bins=50, kde=True)
6 plt.title('Distribución de la Cantidad Vendida (kilo)')
7 plt.xlabel('Cantidad Vendida (kilo)')
8 plt.ylabel('Frecuencia')
9 plt.show()
```

Extracto de código IV.6: Histograma de cantidad vendida

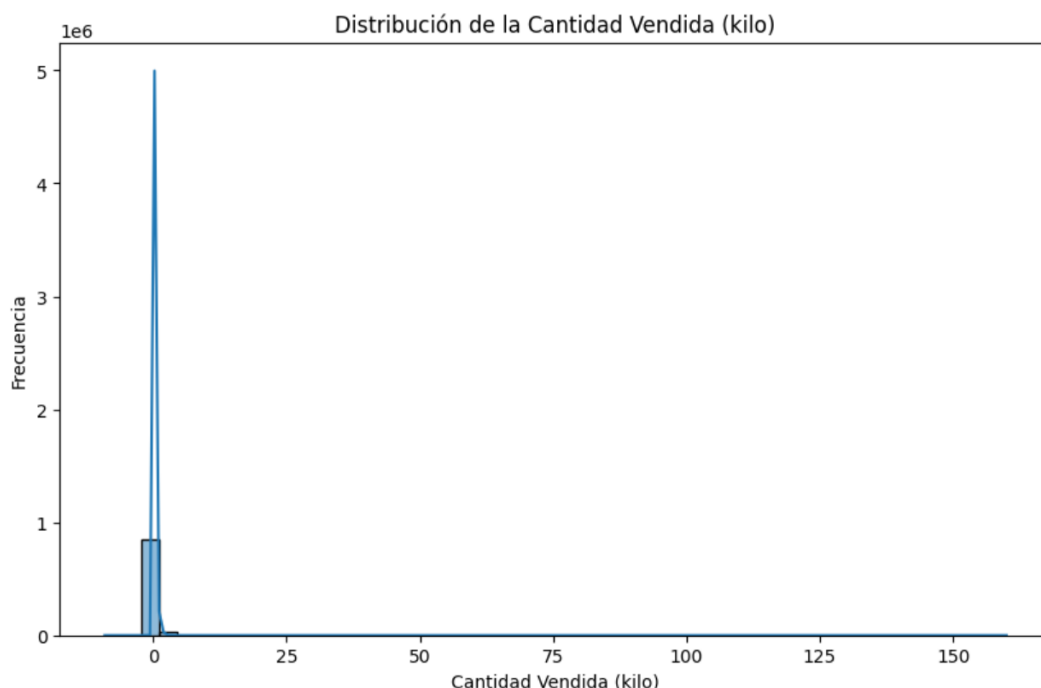


Figura 59: Histograma de la cantidad vendida (kilo)

Distribución del precio de venta

También se analiza cómo se distribuyen los precios unitarios (en EUR/kg) para entender si hay productos muy caros o muy baratos, y ver si la mayoría se encuentra

dentro de un rango determinado.

```
1 plt.figure(figsize=(10, 6))
2 sns.histplot(df_merged['Unit Selling Price (EUR/kg)'], bins=50, kde=True)
3 plt.title('Distribución del Precio de Venta Unitario (EUR/kg)')
4 plt.xlabel('Precio de Venta Unitario (EUR/kg)')
5 plt.ylabel('Frecuencia')
6 plt.show()
```

Extracto de código IV.7: Histograma del precio de venta unitario

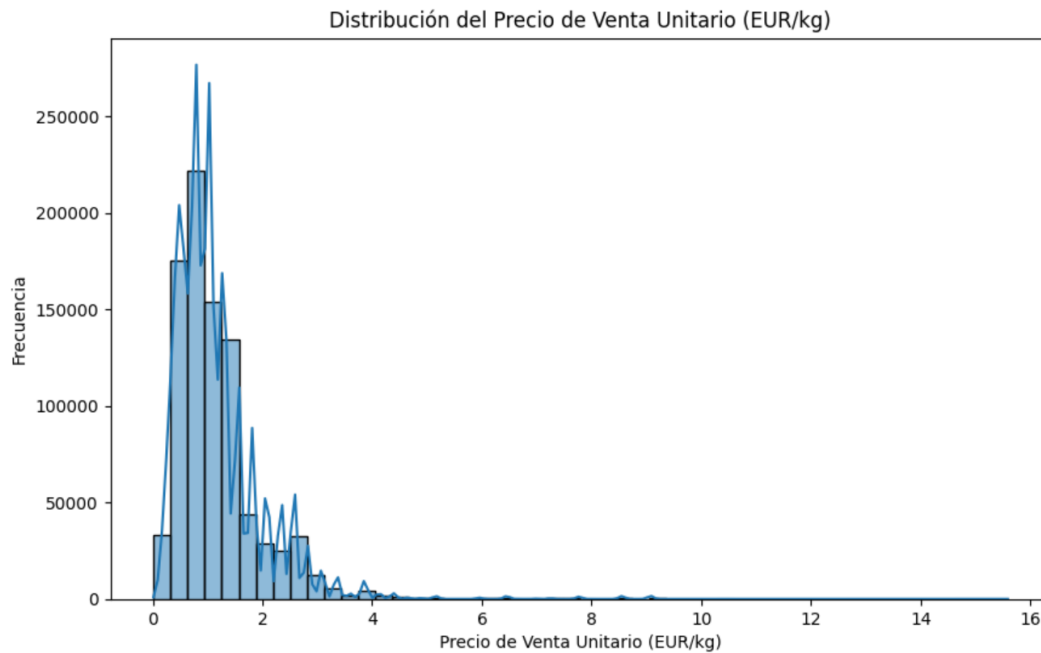


Figura 60: Histograma del precio unitario en EUR/kg

Relación entre cantidad vendida y precio unitario

Se usa un diagrama de dispersión para ver si existe relación entre el precio de un producto y la cantidad que se vende. Esto puede dar pistas sobre si los productos más baratos se venden más o no hay una tendencia clara.

```
1 plt.figure(figsize=(10, 6))
2 sns.scatterplot(x='Quantity Sold (kilo)', y='Unit Selling Price (EUR/kg)', data=
3 df_merged)
4 plt.title('Cantidad Vendida (kilo) vs Precio de Venta Unitario (EUR/kg)')
5 plt.xlabel('Cantidad Vendida (kilo)')
6 plt.ylabel('Precio de Venta Unitario (EUR/kg)')
7 plt.show()
```

Extracto de código IV.8: Scatterplot cantidad vs precio

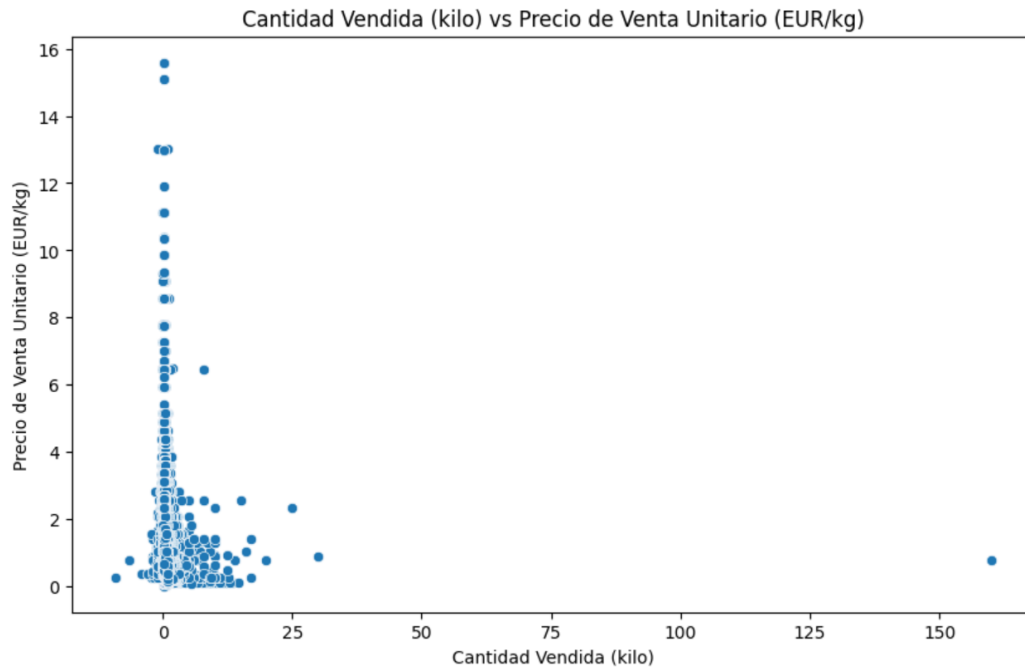


Figura 61: Relación entre cantidad vendida y precio

Ingresos por categoría

Este gráfico permite ver qué categorías de productos generan más ingresos. Es útil para saber cuáles son las más rentables o las que tienen mayor volumen de ventas en euros.

```
1 plt.figure(figsize=(14, 8))
2 sns.barplot(x='Category Name', y='total_sales_revenue', data=category_sales.sort_values(
   'total_sales_revenue', ascending=False))
3 plt.title('Ingresos Totales por Categoría')
4 plt.xlabel('Nombre de Categoría')
5 plt.ylabel('Ingresos Totales (EUR)')
6 plt.xticks(rotation=45, ha='right')
7 plt.tight_layout()
8 plt.show()
```

Extracto de código IV.9: Ingresos por categoría

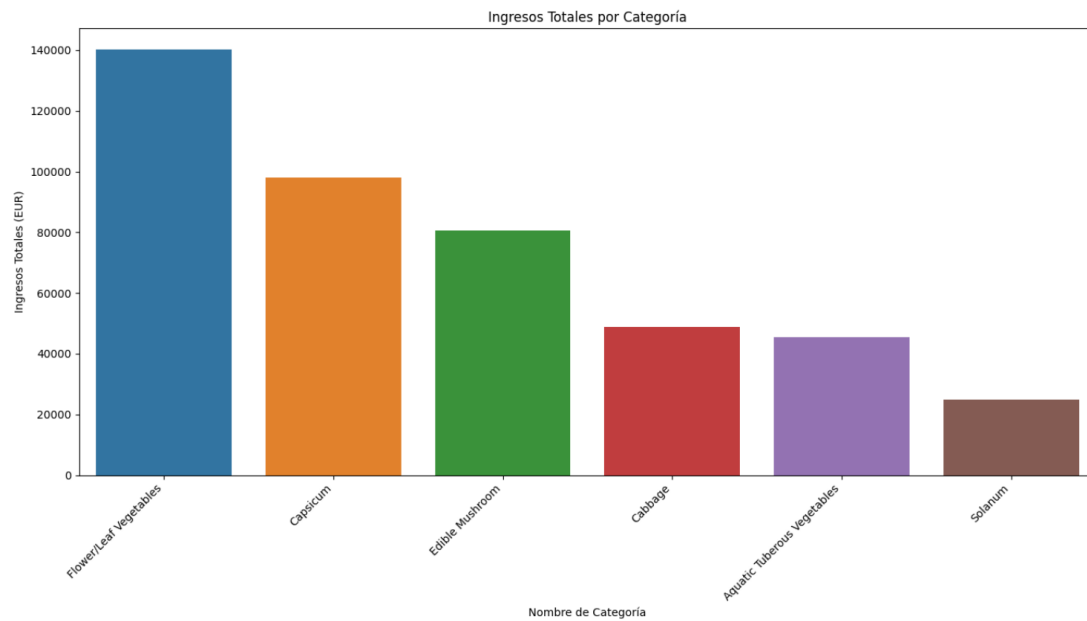


Figura 62: Ingresos totales por categoría

Evolución temporal de ingresos

Finalmente, se representa un gráfico de líneas con los ingresos totales diarios. Esto sirve para detectar estacionalidades, picos de venta o periodos con bajo rendimiento económico.

```

1 plt.figure(figsize=(14, 7))
2 sns.lineplot(x='Date', y='total_sales_revenue', data=daily_trends)
3 plt.title('Ingresos Totales Diarios a lo Largo del Tiempo')
4 plt.xlabel('Fecha')
5 plt.ylabel('Ingresos Totales (EUR)')
6 plt.xticks(rotation=45)
7 plt.tight_layout()
8 plt.show()

```

Extracto de código IV.10: Evolución de ingresos diarios

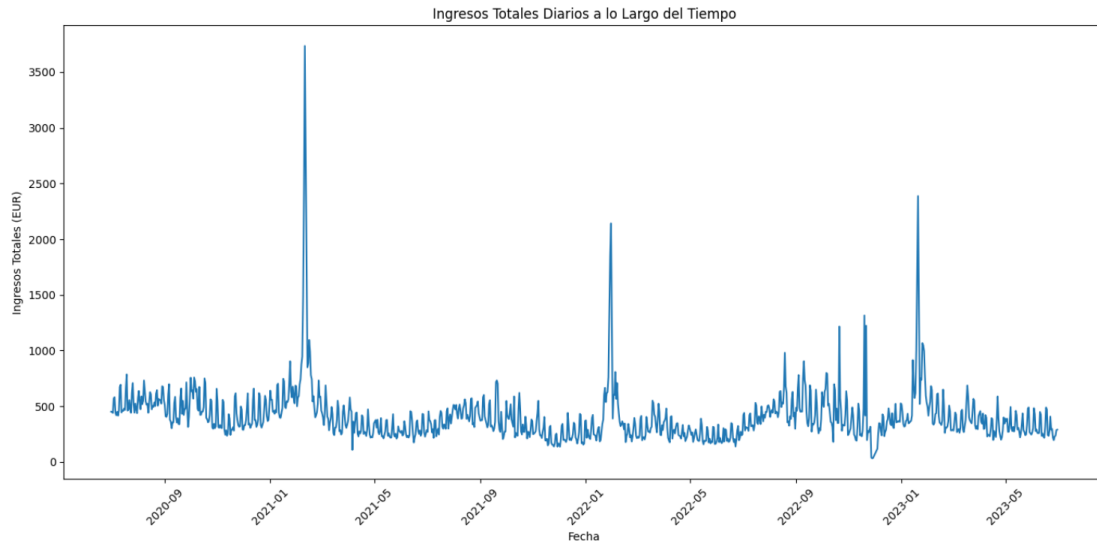


Figura 63: Evolución diaria de los ingresos

1.7. Conclusiones EDA

Gracias a todo el análisis realizado hasta este punto, se ha podido obtener una visión bastante clara sobre cómo se comportan los datos y qué información útil se puede extraer de ellos. Esto es muy importante para poder avanzar con seguridad hacia las siguientes fases del proyecto.

Por un lado, el dataset está bien estructurado y no presenta problemas graves de calidad, ya que no hay muchos valores nulos ni duplicados. Esto ha permitido trabajar directamente con los datos sin necesidad de aplicar una limpieza compleja.

También se ha visto que la mayoría de los productos pertenecen a unas pocas categorías, lo que puede indicar que el negocio tiene una oferta concentrada o que hay una demanda más fuerte en ciertos grupos de productos. A nivel de precios, la mayoría de los valores se mueven en un rango medio, aunque hay algunos extremos que pueden ser productos especiales o errores puntuales.

La relación entre precio y cantidad vendida no es muy clara, lo que sugiere que hay otros factores influyendo, como la categoría del producto o si tiene descuento. Sin embargo, el análisis de ingresos por categoría ha sido muy útil para identificar qué grupos de productos generan más beneficios, lo cual será útil para enfocar estrategias de negocio o para priorizar en futuras predicciones.

Por último, el análisis temporal ha revelado algunos picos de ingresos en fechas concretas. Esto podría deberse a campañas, temporadas o hábitos de compra del cliente. Detectar este tipo de patrones puede ser muy útil para planificar mejor el stock y anticiparse a la demanda.

En resumen, el EDA ha sido fundamental para entender el contexto de los datos y ha aportado información valiosa que se utilizará más adelante en la fase de modelado y predicción. Además, este análisis exploratorio ha permitido identificar

los aspectos clave que deben mostrarse en la interfaz de usuario desarrollada en la aplicación web.

2. Desarrollo de la aplicación

2.1. Arquitectura y diseño de la interfaz

El sistema desarrollado se estructura siguiendo una arquitectura basada en cliente-servidor, donde se diferencian dos componentes principales:

- **Frontend:** Implementado con *React*, es el encargado de mostrar la interfaz de usuario. Desde aquí se envían las peticiones http al **Backend** y se reciben los datos procesados, las visualizaciones y las predicciones.
- **Backend:** Desarrollado con *Flask*, expone una API REST que permite la comunicación entre la interfaz y la lógica de predicción. Las visualizaciones se han realizado con *Bokeh*. Recibe las solicitudes del cliente, procesa los datos y devuelve los resultados.

La comunicación entre frontend y backend se realiza mediante peticiones HTTP en formato JSON. El usuario no interactúa directamente con los datos, ya que el sistema está diseñado para utilizar un único conjunto de datos interno. Esto simplifica el flujo y garantiza la consistencia en la evaluación del modelo.

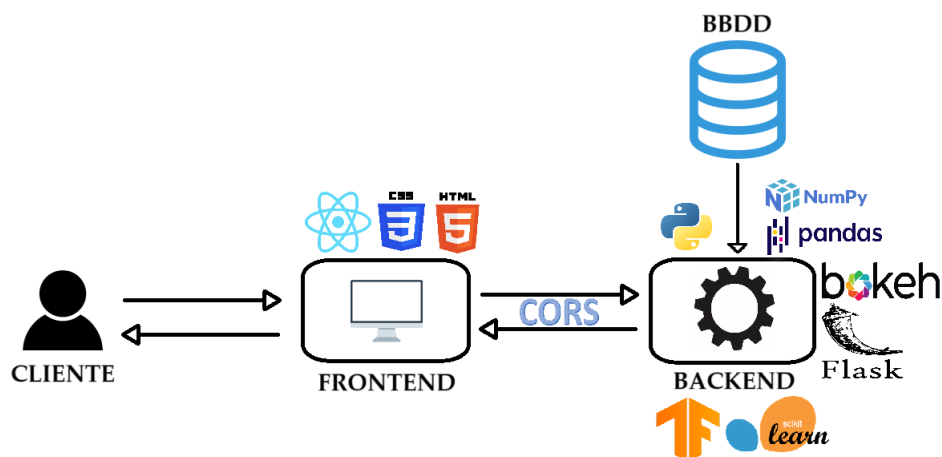


Figura 64: Arquitectura del sistema

La interfaz de usuario ha sido diseñada con un enfoque minimalista, priorizando la claridad y facilidad de interpretación de los resultados. Las principales características son:

- Visualización de los datos históricos y de las predicciones generadas mediante distintos gráficos.

- Presentación de métricas de evaluación numéricas para facilitar la interpretación del rendimiento del modelo.
- Navegación sencilla y sin elementos innecesarios, ya que el sistema no requiere interacción activa por parte del usuario (por ejemplo, no existe un formulario de carga de datos).

A continuación se presentan las figuras que representan el diseño de la interfaz descrito anteriormente.

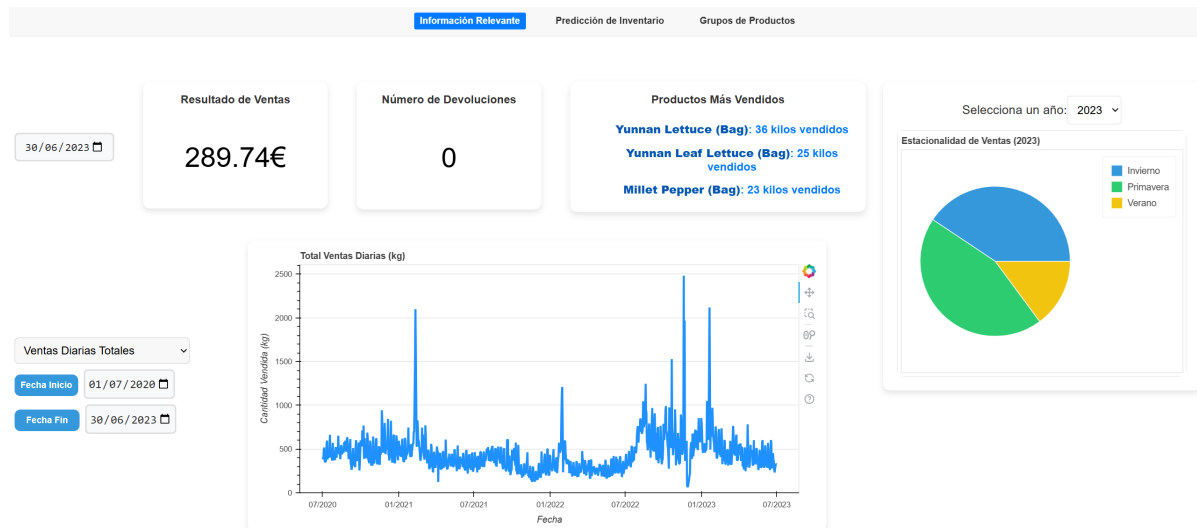


Figura 65: Diseño de la página principal de la aplicación web

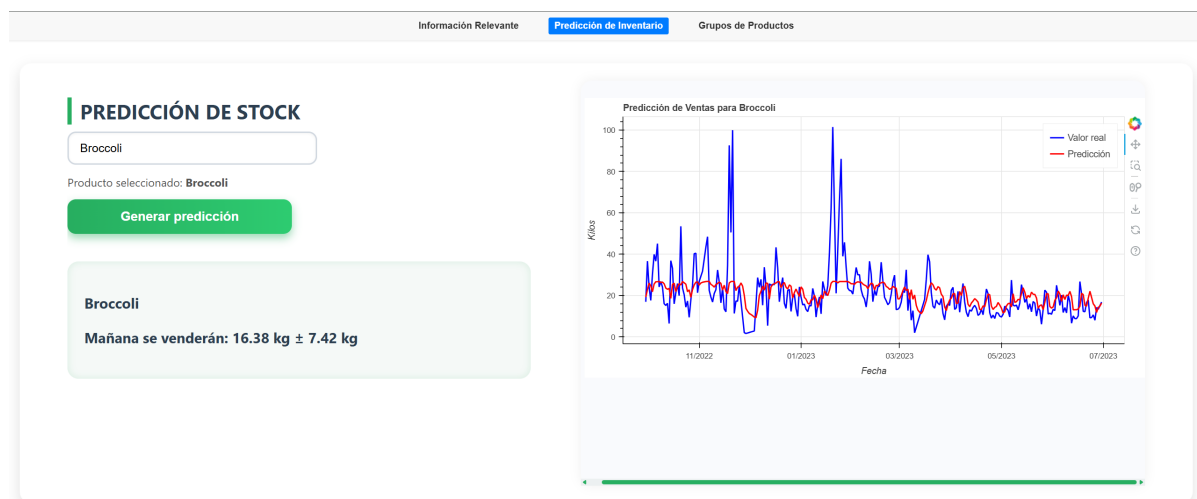


Figura 66: Diseño de la página de previsión de inventario de la aplicación web



Figura 67: Diseño de la página de grupos de productos de la aplicación web

2.2. Manual de Usuario

El manual de usuario contiene la explicación de todos los detalles respecto a la instalación y el uso correcto de la solución implementada. El proyecto se encuentra alojado en la dirección: <https://github.com/sanmingar1/Trabajo-Fin-de-Grado>.

Para poder ejecutar correctamente la aplicación es necesario ejecutar el archivo `instalacion_dependencias.bat` una única vez por dispositivo. Este archivo contiene un script que crea e inicia un entorno virtual donde posteriormente se instalan los componentes necesarios para que la aplicación web funcione correctamente.

Una vez completada esta instalación, la aplicación puede iniciarse mediante el archivo `iniciar.bat`, que abre dos ventanas de la consola de comandos y ejecuta los comandos necesarios para poner en marcha la aplicación en local.

Tras realizar estos pasos, el usuario tendrá acceso a toda la funcionalidad de la aplicación desde el puerto 3000 del sistema. A partir de ahí podrá utilizar todas las características que ofrece la aplicación.

2.3. Desarrollo del Backend

Introducción

El *backend* de la aplicación constituye el núcleo del sistema de análisis de datos de ventas, implementado utilizando **Flask** como framework web principal. Esta arquitectura permite procesar grandes volúmenes de datos y ofrecer análisis estadísticos y predictivos mediante una API REST robusta y escalable.

Arquitectura del Sistema

Framework Principal: Flask Se eligió Flask debido a su simplicidad, flexibilidad y capacidad para manejar APIs REST de forma eficiente. Flask permite una implementación modular, donde cada funcionalidad analítica se encapsula en módulos independientes, lo que facilita el mantenimiento y la escalabilidad del sistema.

Gestión de CORS La integración de Flask-CORS permite la comunicación entre el frontend y el backend incluso cuando se ejecutan en distintos puertos, eliminando restricciones de origen cruzado (*same-origin policy*) y facilitando la integración con aplicaciones web modernas.

Estructura Modular

El backend se organiza en módulos especializados, cada uno responsable de una funcionalidad específica:

Módulo de Ventas por Fecha (`ventas_fecha.py`) Calcula los ingresos generados en una fecha específica, multiplicando el precio unitario por la cantidad vendida. Incluye validación de fechas y manejo de errores para garantizar la integridad de los resultados.

Módulo de Productos Más Vendidos (`productos_mas_vendidos.py`) Identifica los productos con mayor volumen de ventas en una fecha dada, utilizando agregaciones eficientes con pandas. Devuelve los resultados en formato JSON estructurado.

Módulo de Análisis de Devoluciones (`numero_devoluciones_fecha.py`) Calcula métricas sobre devoluciones de productos, filtrando por tipo y fecha. Es útil para el análisis de calidad y satisfacción del cliente.

Módulo de Estacionalidad (`estacionalidad_ventas.py`) Analiza patrones estacionales clasificando ventas por estaciones del año. Genera visualizaciones interactivas mediante Bokeh.

Módulo de Clustering (`clustering.py`) Aplica K-means para agrupar transacciones según patrones de compra. Utiliza matrices de co-ocurrencia y permite identificar cestas de compra frecuentes.

Módulo de Predicción de Stock (`prediccion_stock.py`) Implementa redes neuronales LSTM para predecir la demanda de productos considerando variables temporales como el día de la semana, mes y fines de semana. Incluye métricas como MAE y RMSE.

Módulo de Visualización Avanzada (`grafica_grande.py`) Genera gráficas complejas con Bokeh, incluyendo análisis de ventas diarias, horarios de compra y rankings de productos, con capacidades de zoom, hover y filtrado temporal.

Tecnologías de Análisis de Datos

Pandas Se emplea para manipulación y análisis de datos estructurados, realizando filtrado, agrupación y agregación sobre los conjuntos de ventas.

Scikit-learn Proporciona algoritmos de aprendizaje automático para tareas como clustering y evaluación, facilitando la segmentación de clientes y análisis de patrones.

TensorFlow/Keras Framework utilizado para implementar los modelos de redes LSTM que permiten predicciones precisas sobre series temporales de ventas.

Bokeh Herramienta de visualización interactiva que permite generar gráficos en HTML/JavaScript, ofreciendo una experiencia visual rica al usuario final.

API REST

La API implementa endpoints RESTful estructurados y bien definidos:

- POST `/api/ventas_fecha`: Cálculo de ingresos por fecha.
- POST `/api/productos_mas_vendidos`: Ranking de productos.
- POST `/api/numero_devoluciones_fecha`: Análisis de devoluciones.
- POST `/api/estacionalidadVentas`: Análisis estacional.
- POST `/api/clustering`: Agrupamiento de productos.
- POST `/api/prediccion_stock1`: Predicción de demanda.
- POST `/api/prueba_grafica_grande`: Visualizaciones avanzadas.
- GET `/api/lista_productos`: Consulta de catálogo de productos.

Procesamiento de Datos

El procesamiento de datos constituye una fase esencial para garantizar que la información utilizada en el análisis y en la aplicación web sea fiable, consistente y eficiente de manejar. En este proyecto se han considerado tres aspectos fundamentales: limpieza y validación, optimización de rendimiento y manejo de errores.

Limpieza y validación Antes de aplicar cualquier modelo predictivo o de visualización, se llevó a cabo un proceso de depuración de los datos. Esto incluyó la eliminación de registros duplicados, la comprobación de la coherencia en los rangos de fechas y la validación de los tipos de datos (por ejemplo, asegurar que los campos numéricos y de fecha no contuvieran valores erróneos). Asimismo, en los análisis de ventas se filtraron las transacciones de devolución cuando podían distorsionar las métricas principales.

Optimización de rendimiento Dado que la aplicación web debe ofrecer respuestas rápidas incluso con volúmenes de datos elevados, se aplicaron varias técnicas de optimización. Entre ellas destacan:

- La indexación temporal, que facilita las consultas basadas en intervalos de fechas.
- El uso de operaciones vectorizadas con la librería pandas, lo que permite realizar agregaciones y transformaciones de forma más eficiente que con bucles tradicionales.

Manejo de errores Finalmente, se implementó un esquema robusto de gestión de errores para garantizar la estabilidad de la aplicación. Esto incluye la validación de parámetros de entrada antes de ejecutar consultas, la detección y notificación de archivos ausentes, y la devolución de códigos de respuesta HTTP adecuados en la API según el tipo de error (por ejemplo, 400 para parámetros inválidos o 404 para recursos no encontrados). De esta forma, se asegura que el sistema no solo sea funcional, sino también tolerante a fallos y fácil de mantener.

Escalabilidad y Mantenimiento

La arquitectura modular permite extender funcionalidades sin afectar las existentes. Cada módulo puede actualizarse, probarse y desplegarse de forma independiente, facilitando el mantenimiento y escalado del sistema.

Consideraciones de Seguridad

Aunque el sistema está orientado al entorno de desarrollo, se han aplicado validaciones básicas de entrada y sanitización para prevenir ataques comunes

como inyecciones. Para despliegues en producción, se recomienda incluir autenticación, autorización y cifrado de datos sensibles.

2.4. Desarrollo del Frontend

Introducción

El desarrollo del *frontend* constituye una parte fundamental del proyecto, ya que proporciona la interfaz de usuario que permite interactuar con las funcionalidades de análisis de datos desarrolladas en el *backend*. Se ha optado por desarrollar una aplicación web interactiva que facilite la visualización y exploración de los datos de ventas de manera intuitiva y eficiente.

Tecnologías Utilizadas

React.js Se ha elegido React.js como biblioteca principal para el desarrollo del *frontend* por las siguientes razones:

- **Componentización:** Permite dividir la interfaz en componentes reutilizables y modulares, facilitando el mantenimiento y la escalabilidad del código.
- **Gestión de estado:** Ofrece herramientas nativas para manejar el estado de la aplicación de manera eficiente mediante *hooks* como `useState` y `useEffect`.
- **Ecosistema maduro:** Cuenta con una amplia comunidad y documentación, lo que facilita la resolución de problemas y la implementación de funcionalidades avanzadas.
- **Renderizado eficiente:** El *Virtual DOM* de React optimiza las actualizaciones de la interfaz, mejorando el rendimiento general de la aplicación.

CSS3 y Diseño Responsive Se ha implementado un sistema de estilos personalizado utilizando CSS3 puro, evitando dependencias externas de frameworks de UI. Esta decisión se basa en:

- **Control total sobre el diseño:** Permite crear una identidad visual única y personalizada para la aplicación.
- **Optimización del rendimiento:** Elimina la carga de librerías CSS externas innecesarias.
- **Flexibilidad:** Facilita la implementación de animaciones y efectos visuales específicos.

Arquitectura de la Aplicación

Estructura de Componentes La aplicación sigue una arquitectura basada en componentes, donde cada funcionalidad está encapsulada en un componente

independiente:

App.js (Componente raíz con navegación)
eda.js (Dashboard principal)
prediccion_stock.js (Módulo de predicción)
clustering.js (Visualización de grupos)

Componentes específicos:

ventas_fecha.js
productos_mas_vendidos.js
numero_devoluciones.js
estacionalidadVentas.js
grafica_grande.js

Gestión de Estado Se utiliza el hook `useState` de React para manejar el estado local de cada componente, mientras que la comunicación entre componentes se realiza mediante el paso de *props*. Esta aproximación mantiene la simplicidad y claridad del código, evitando la complejidad de gestores de estado externos como Redux.

Funcionalidades Implementadas

Dashboard Principal (EDA) El componente `eda.js` actúa como el panel principal de la aplicación, presentando múltiples métricas y visualizaciones:

- **Selector de fecha centralizado:** Permite filtrar todos los datos por una fecha específica.
- **Métricas clave:** Muestra ventas diarias, número de devoluciones y productos más vendidos.
- **Gráficas interactivas:** Integra visualizaciones generadas por Bokeh desde el *backend*.
- **Layout responsive:** Organiza los elementos en un diseño adaptable a diferentes tamaños de pantalla.

Predicción de Stock El módulo de predicción (`prediccion_stock.js`) implementa una interfaz intuitiva para generar predicciones de inventario:

- **Buscador con autocompletado:** Permite buscar productos con sugerencias en tiempo real.
- **Validación de entrada:** Filtra caracteres especiales y valida la selección del usuario.
- **Visualización de resultados:** Muestra tanto la predicción numérica como un gráfico interactivo.
- **Gestión de estados de carga:** Proporciona *feedback* visual durante el procesamiento.

Análisis de Clustering El componente `clustering.js` presenta los resultados de la agrupación de productos:

- **Visualización por grupos:** Organiza los productos en tarjetas categorizadas.
- **Diseño adaptativo:** Utiliza CSS Grid para una distribución óptima de los grupos.
- **Interactividad:** Efectos *hover* y transiciones suaves para mejorar la experiencia de usuario.

Visualizaciones Interactivas Se ha implementado la integración con Bokeh para renderizar gráficos avanzados:

- **Gráficas de series temporales:** Para análisis de tendencias de ventas.
- **Gráficos de barras:** Para comparación de productos más vendidos.
- **Visualizaciones de estacionalidad:** Gráfico circular para identificar patrones estacionales.

Comunicación con el Backend

API REST La comunicación entre el *frontend* y el *backend* se realiza mediante una API REST desarrollada con Flask:

- **Endpoints estructurados:** Cada funcionalidad tiene su endpoint específico.
- **Protocolo HTTP:** Utiliza métodos POST para enviar parámetros y recibir datos.
- **Manejo de errores:** Implementa bloques try-catch para gestionar errores de red y del servidor.
- **Formato JSON:** Todas las comunicaciones utilizan JSON como formato de intercambio.

Integración con Bokeh Para las visualizaciones avanzadas, se utiliza la biblioteca Bokeh del *backend*:

- **Embed dinámico:** Los gráficos se generan en el servidor y se insertan en el *frontend*.
- **Interactividad preservada:** Se mantienen las funcionalidades interactivas originales de Bokeh.
- **Actualización en tiempo real:** Los gráficos se actualizan automáticamente al modificar parámetros.

Diseño de la Interfaz de Usuario

Principios de Diseño El diseño de la interfaz sigue principios de usabilidad y experiencia de usuario:

- **Jerarquía visual clara:** Uso de tamaños de fuente, colores y espaciado para guiar la atención.
- **Consistencia:** Patrones de diseño coherentes en toda la aplicación.
- **Accesibilidad:** Contrastes adecuados y navegación por teclado.
- **Feedback visual:** Retroalimentación inmediata ante las acciones del usuario.

Paleta de Colores y Tipografía

- **Colores principales:** Tonos de azul (#007bff, #2e86de) para elementos interactivos.
- **Colores secundarios:** Grises para texto y componentes de soporte.
- **Tipografía:** Arial y Segoe UI, elegidas por su legibilidad en distintos dispositivos.
- **Espaciado:** Uso de unidades relativas (vh/vw) para diseño responsive.

Optimización y Rendimiento

El desarrollo de la aplicación consideró principios de eficiencia y optimización, con el fin de garantizar tiempos de respuesta adecuados y una interacción fluida. Para ello, se aplicaron técnicas de carga diferida de componentes, reduciendo el tiempo inicial de arranque, y se optimizó el ciclo de renderizado de React, de manera que solo se actualicen los elementos necesarios en cada interacción. Asimismo, en los mecanismos de búsqueda se introdujo un retardo controlado en la ejecución de consultas, lo que disminuye la carga sobre el servidor y mejora la experiencia de usuario. Estas medidas, junto con la compresión de recursos estáticos, responden a recomendaciones habituales en el diseño de aplicaciones web modernas

En cuanto al uso de memoria, se incluyó un control explícito de la liberación de recursos cuando los componentes dejan de estar activos, evitando fugas de memoria y garantizando un rendimiento estable a lo largo del tiempo. Este enfoque se alinea con buenas prácticas de ingeniería de software orientadas a la fiabilidad y la eficiencia.

Responsividad y Adaptabilidad

La interfaz se diseñó siguiendo criterios de diseño responsivo, de modo que la aplicación pueda utilizarse de manera consistente en dispositivos con diferentes tamaños de pantalla. Para ello se emplearon estructuras flexibles en la maquetación y estilos adaptativos, lo que asegura una correcta visualización de la información y

contribuye a la accesibilidad del sistema. Esta decisión se fundamenta en la necesidad de ofrecer una experiencia de usuario homogénea, independientemente del dispositivo utilizado.

Mantenibilidad y Escalabilidad

Finalmente, la arquitectura del sistema se planteó con el objetivo de favorecer la mantenibilidad y la escalabilidad. Los componentes fueron diseñados con responsabilidades bien delimitadas, promoviendo su reutilización en distintos contextos de la aplicación. Asimismo, la organización de archivos y módulos se realizó de forma modular y coherente, facilitando la incorporación de nuevas funcionalidades sin comprometer la estabilidad del sistema. Estas prácticas responden a principios de diseño orientado a componentes y a criterios de calidad de software recogidos en normas como ISO/IEC 25010.

Conclusiones

El desarrollo del *frontend* dio como resultado una aplicación web que cumple con los objetivos planteados y que se adapta a las necesidades específicas del proyecto. La elección de React.js como biblioteca permitió implementar las funcionalidades requeridas con un alto grado de flexibilidad, manteniendo al mismo tiempo una estructura clara y modular que facilita su mantenimiento y posible ampliación en el futuro.

La interfaz fue diseñada bajo un enfoque responsivo, lo que garantiza su correcta visualización en distintos dispositivos, y se integró de forma eficiente con el *backend* desarrollado en Flask. Además, se aplicaron técnicas de optimización orientadas a la reducción de tiempos de carga y a la estabilidad en la gestión de datos, lo que asegura un rendimiento adecuado incluso al trabajar con volúmenes de información elevados. Con ello, se cumplen tanto los requisitos funcionales como los criterios técnicos definidos para el sistema.

3. Investigación y Predicción

En esta sección se plantea un estudio comparativo entre distintos modelos de predicción para series temporales. El objetivo es evaluar el rendimiento de varios algoritmos supervisados —incluyendo regresores lineales, modelos basados en árboles y redes neuronales LSTM— aplicados a la previsión de valores futuros en una secuencia temporal univariada y multivariada. La investigación se centra en identificar qué enfoque ofrece menor error de predicción bajo diferentes configuraciones y conjuntos de variables, utilizando métricas estándar como MAE y RMSE.

Los experimentos realizados muestran que, cuando se trabaja únicamente con la serie de ventas, los modelos clásicos (como la regresión lineal) ofrecen resultados

competitivos y en ocasiones superiores a los de las redes neuronales. Sin embargo, al incorporar información adicional —como precios o variables de calendario— el modelo LSTM consigue una mejora notable en la capacidad predictiva, reduciendo significativamente los errores frente a los modelos basados en árboles o regresión. El modelo Random Forest, configurado con 100 árboles, también mostró un buen desempeño, situándose como alternativa robusta especialmente en escenarios multivariados.

Estos hallazgos no solo tienen un valor académico, sino que también se integran directamente en la aplicación web desarrollada. En la interfaz de usuario, los resultados de predicción se presentan de manera gráfica junto con las ventas históricas, permitiendo al usuario visualizar tendencias y comparar valores reales con los estimados por el modelo. Además, la arquitectura del sistema facilita la selección del algoritmo más adecuado para cada producto: la API expone los resultados del modelo entrenado y la aplicación cliente los representa en dashboards interactivos. De este modo, la investigación realizada se convierte en una funcionalidad tangible para el negocio, aportando valor práctico a la toma de decisiones.

3.1. Metodología y Enfoque Experimental

La investigación desarrollada en este trabajo se centra en la predicción de ventas de productos agrícolas mediante el análisis de series temporales, utilizando como caso de estudio las ventas del producto *Amaranth*. El enfoque metodológico adoptado sigue una progresión sistemática que permite evaluar la contribución de diferentes tipos de información en la capacidad predictiva de los modelos.

Para la validación de resultados se realizó una separación temporal de los datos, garantizando que las observaciones destinadas a prueba correspondieran siempre a fechas posteriores a las de entrenamiento. En los experimentos univariantes se utilizó un 70 % de los datos para entrenamiento y un 30 % para prueba, mientras que en los multivariantes se reservaron un 75 % para entrenamiento y un 25 % para prueba. En otros experimentos con variables adicionales (por ejemplo, precios), la proporción de test alcanzó hasta un 40 %. En todos los casos, se aplicó una ventana deslizante de 7 días para construir las variables de entrada y un horizonte de predicción de 1 día.

En el caso de la red LSTM, el modelo estuvo formado por una capa con 64 unidades y activación tanh, seguida de una capa de salida densa. Se entrenó durante 20 épocas con un tamaño de batch de 32, utilizando el optimizador Adam con tasa de aprendizaje de 0.001 y un 10 % de los datos de entrenamiento reservados para validación. La entrada se organizó en secuencias de 7 días consecutivos y la salida correspondió a la predicción del día siguiente. Estos experimentos se realizaron tanto en configuración univariante (solo con ventas) como multivariante (incorporando variables como precio o calendario).

Por su parte, el modelo de *Random Forest* se configuró con 100 árboles de decisión (`n_estimators=100`) y una semilla de aleatoriedad fijada en 42 para

asegurar la reproducibilidad. Este modelo se evaluó bajo las mismas divisiones temporales de entrenamiento y prueba, incluyendo también versiones con variables adicionales como precio o calendario. La comparación se completó con otros modelos de referencia como regresión lineal y XGBoost, lo que permitió valorar la capacidad relativa de cada enfoque.

Estrategia de Modelado Progresivo

La investigación se estructura siguiendo una metodología de complejidad creciente, comenzando con enfoques simples y evolucionando hacia modelos más sofisticados. Esta estrategia permite identificar qué componentes de información aportan mayor valor predictivo y justificar la necesidad de modelos más complejos.

Regresión Univariante: Fundamento Base El primer enfoque experimental se basa en la regresión univariante, utilizando exclusivamente la serie temporal de kilogramos vendidos como variable predictora. Esta aproximación establece una línea base fundamental para evaluar el comportamiento intrínseco de la serie temporal. Se implementaron tres algoritmos de aprendizaje automático: regresión lineal, Random Forest y XGBoost, utilizando una ventana deslizante de siete días para predecir las ventas del día siguiente.

Los resultados obtenidos muestran que la regresión lineal alcanza el mejor rendimiento con un MAE de 1.46 y RMSE de 2.33, seguida de cerca por Random Forest (MAE: 1.49, RMSE: 2.44). Esta superioridad de la regresión lineal sugiere que las relaciones temporales en los datos pueden ser capturadas efectivamente mediante aproximaciones lineales simples, lo cual es un hallazgo relevante que indica la presencia de patrones temporales consistentes.

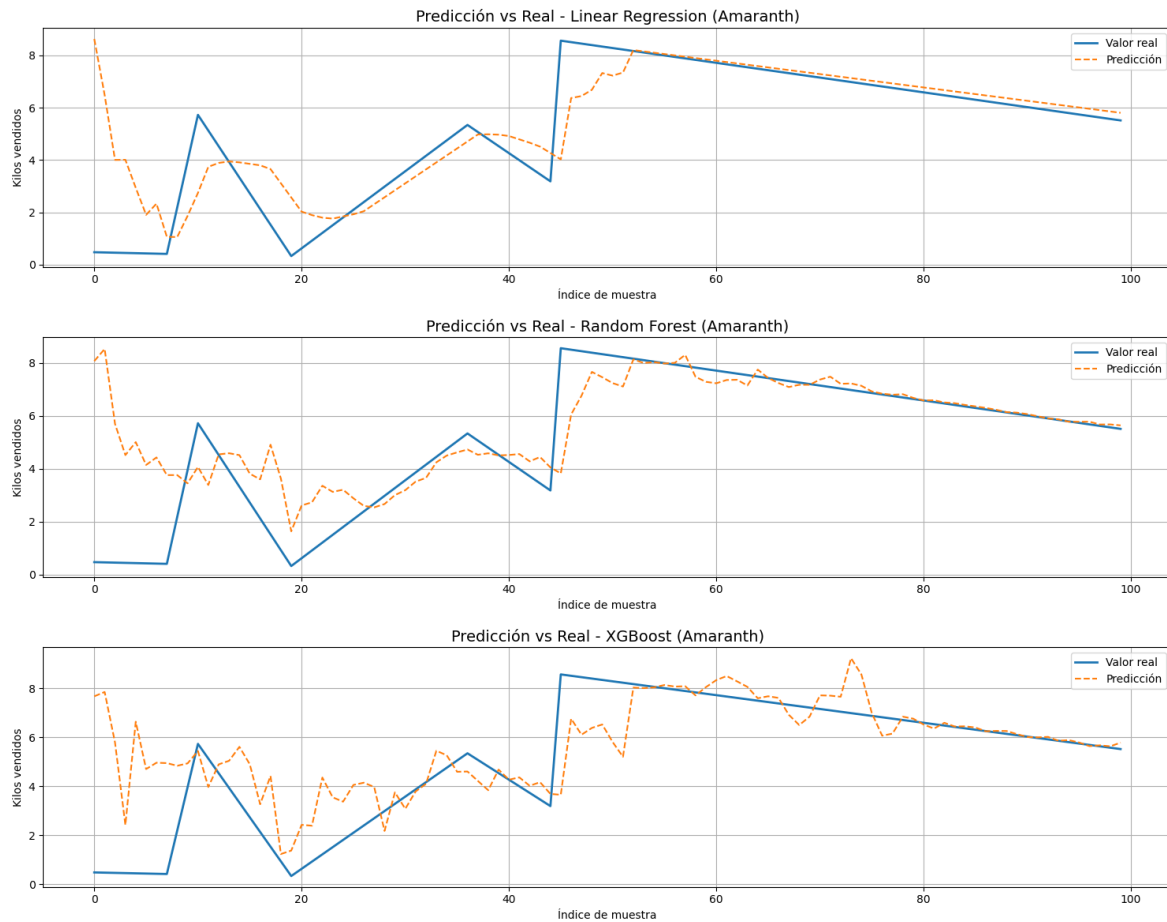


Figura 68: Resultados de la regresión univariante

Incorporación de Información de Precios: Regresión Bivariada El segundo experimento incorpora el precio por kilogramo como variable adicional, bajo la hipótesis de que existe una relación entre el precio y la demanda que puede mejorar las predicciones. Esta decisión se fundamenta en principios económicos básicos donde la elasticidad precio-demanda influye en los patrones de venta.

Sin embargo, los resultados revelan una degradación en el rendimiento predictivo, con la regresión lineal alcanzando un MAE de 1.87 y RMSE de 3.04. Este deterioro sugiere que la relación precio-demanda en el contexto específico de este producto puede ser más compleja de lo esperado, o que la información de precios introduce ruido que los modelos no logran procesar efectivamente con la arquitectura utilizada.

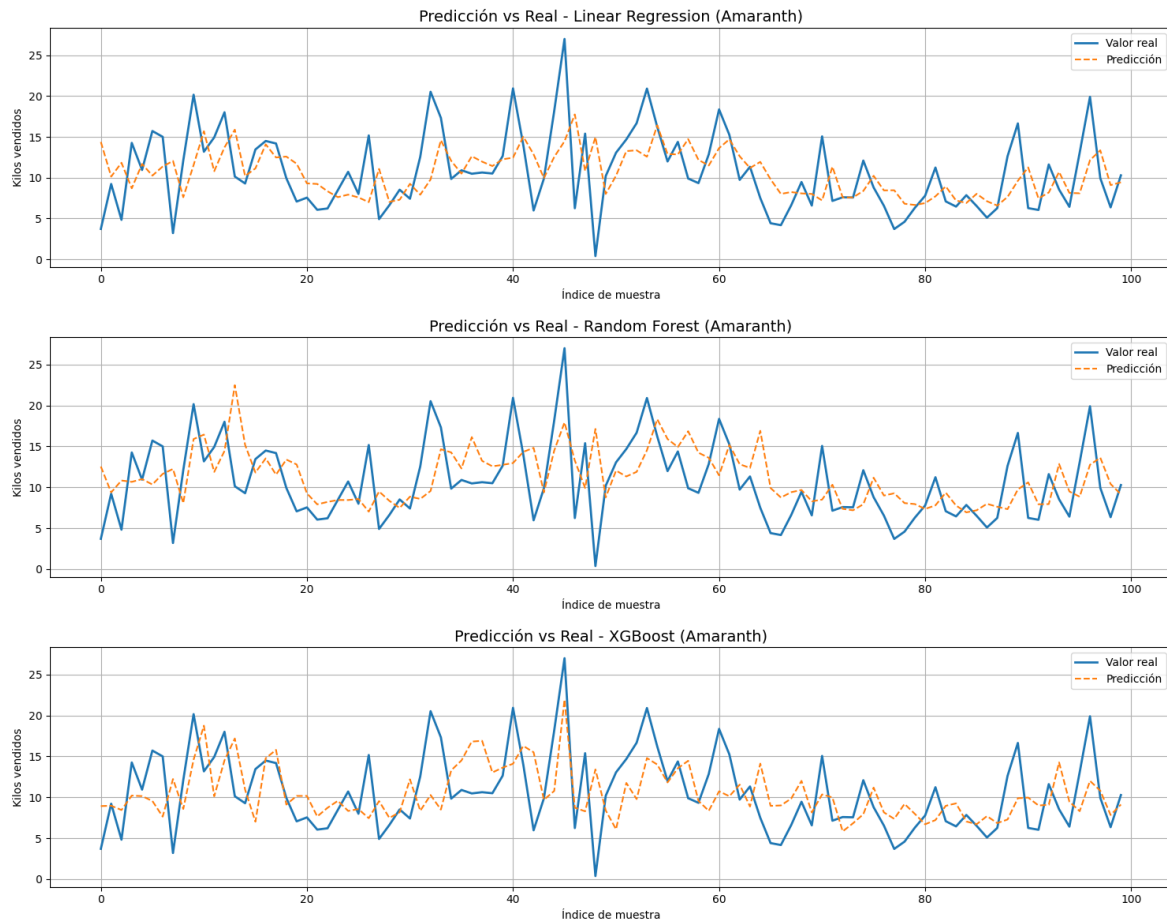


Figura 69: Resultados de la regresión bivariada

Enriquecimiento con Variables Temporales: Regresión Multivariante La tercera fase experimental introduce características temporales derivadas de las fechas: día de la semana, mes, día del año y una variable binaria indicando si es fin de semana. Esta aproximación se basa en el conocimiento domain de que los patrones de compra de productos agrícolas suelen mostrar estacionalidades y ciclicidades temporales.

Los resultados muestran una mejora notable, particularmente con Random Forest que alcanza un MAE de 1.73 y RMSE de 2.70. Esta mejora indica que los patrones temporales cíclicos son efectivamente relevantes para la predicción, y que los modelos basados en árboles como Random Forest pueden capturar mejor estas relaciones no lineales entre las variables temporales y las ventas.

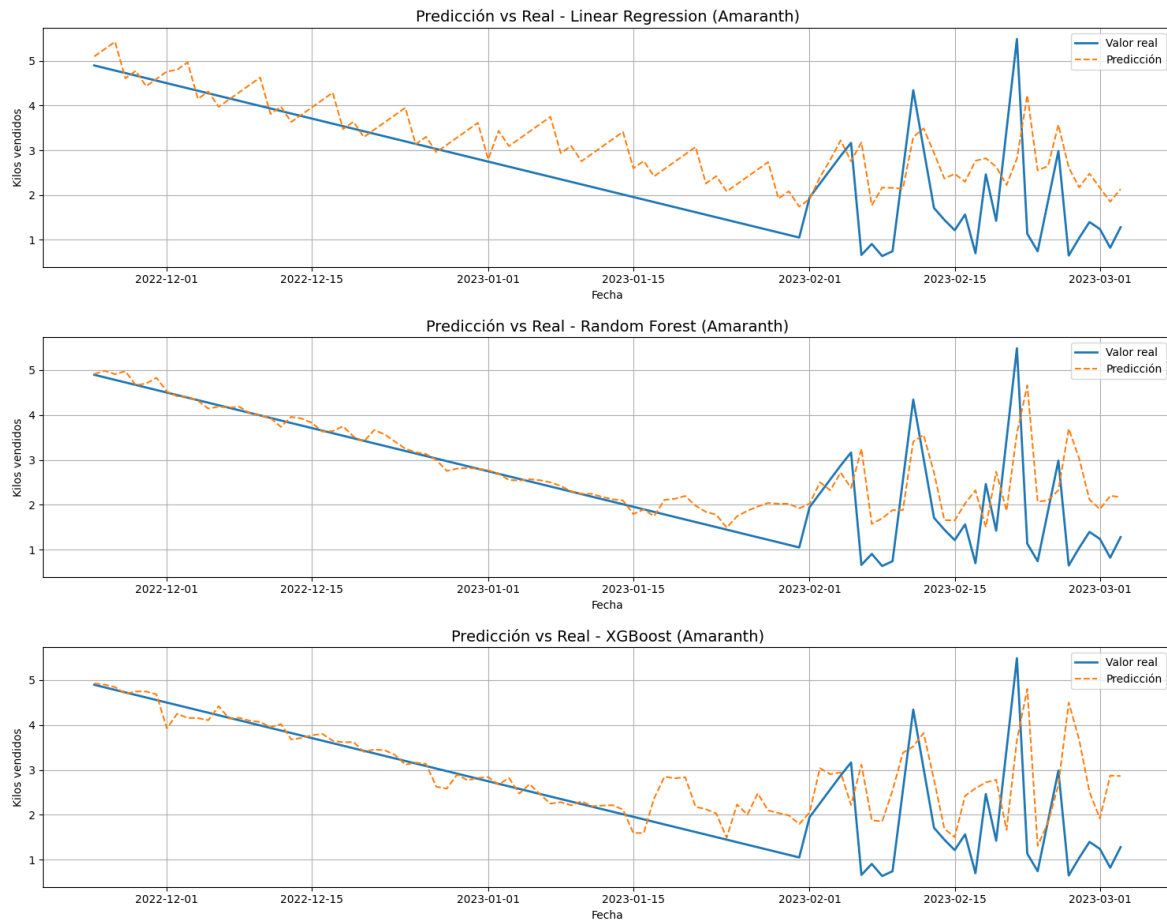


Figura 70: Resultados de la regresión multivariante

Implementación de Redes Neuronales Recurrentes: Modelos LSTM

La segunda parte de la investigación explora el uso de redes neuronales LSTM (Long Short-Term Memory), especialmente diseñadas para el procesamiento de secuencias temporales. Esta elección se justifica por la capacidad teórica de las LSTM para capturar dependencias temporales de largo plazo que los modelos tradicionales podrían no detectar.

LSTM Univariante: Procesamiento Secuencial Básico El modelo LSTM univariante procesa secuencias de siete días de datos de ventas para predecir el día siguiente. Con una arquitectura de 64 unidades LSTM y entrenamiento durante 20 épocas, el modelo alcanza un MAE de 1.56 y RMSE de 2.42. Estos resultados son comparables a los mejores modelos tradicionales, sugiriendo que las LSTM pueden capturar efectivamente los patrones temporales, aunque sin una ventaja clara sobre métodos más simples.

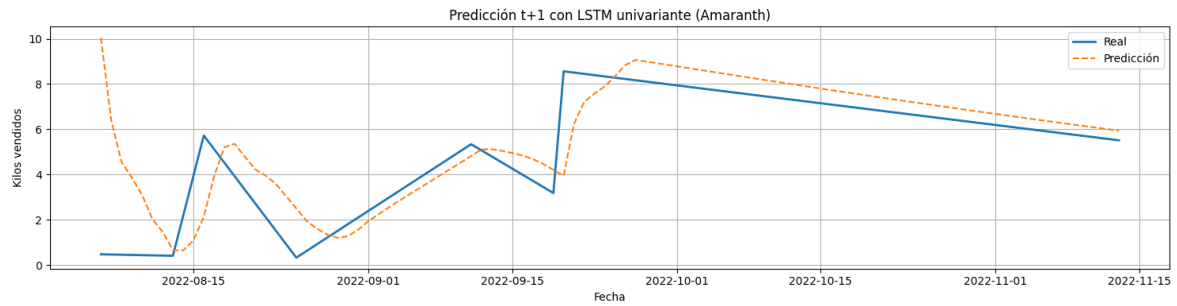


Figura 71: Resultados del modelo LSTM univariante

LSTM Multivariante: Integración de Información Temporal La incorporación de variables temporales en la arquitectura LSTM mejora significativamente el rendimiento, alcanzando un MAE de 1.43 y RMSE de 2.30. Esta mejora demuestra que las redes neuronales pueden procesar efectivamente información multidimensional temporal, aprovechando tanto los patrones secuenciales como las características cíclicas.

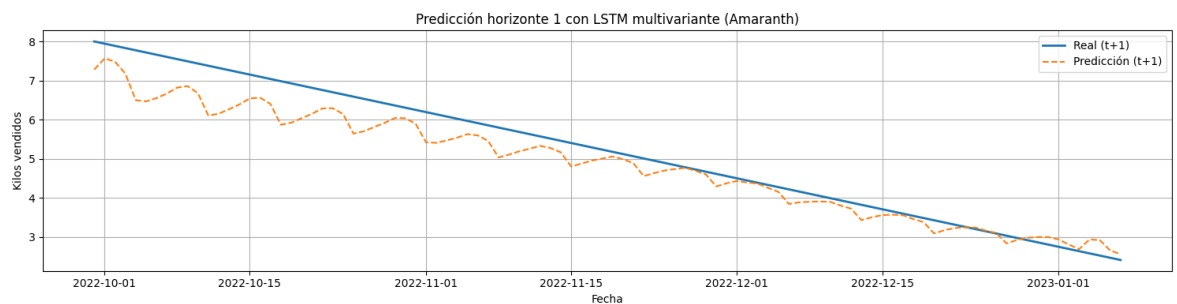


Figura 72: Resultados del modelo LSTM multivariante

LSTM bivariado: procesamiento de precio y demanda El modelo LSTM que incorpora información de precios mantiene un rendimiento competitivo (MAE: 1.40, RMSE: 2.36), mostrando que las redes neuronales pueden manejar mejor la complejidad de la relación precio–demanda en comparación con los modelos tradicionales. En este caso, el término *bivariado* hace referencia a que el modelo no solo utiliza la serie temporal de ventas (demanda), sino también una segunda variable explicativa (precio), lo que permite capturar de manera más precisa la interacción entre ambas.

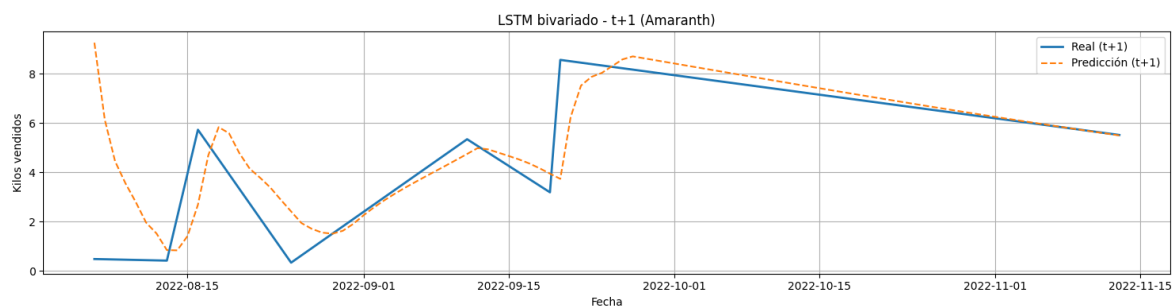


Figura 73: Resultados del modelo LSTM bivariado

Tabla IV.1: Comparación de rendimiento de modelos: MAE y RMSE

Tipo de Modelo	Modelo	MAE	RMSE
Regresión Univariante	Linear Regression	1.46	2.33
	Random Forest	1.49	2.44
	XGBoost	1.72	2.72
Regresión Bivariada (con precio/kg)	Linear Regression	1.87	3.04
	Random Forest	2.23	3.37
	XGBoost	2.46	3.60
Regresión Multivariante (con variables temporales)	Linear Regression	1.82	2.66
	Random Forest	1.73	2.70
	XGBoost	1.86	2.99
Modelos LSTM	LSTM Univariante	1.56	2.42
	LSTM Multivariante	1.43	2.30
	LSTM Bivariado	1.40	2.36

Justificación de la Metodología

La progresión metodológica adoptada permite varias conclusiones importantes. Primero, establece que los patrones temporales intrínsecos de las ventas son suficientemente consistentes para permitir predicciones razonablemente precisas con modelos simples. Segundo, demuestra que la información temporal contextual (día de la semana, estacionalidad) aporta valor predictivo significativo. Tercero, revela que la relación precio-demanda requiere enfoques más sofisticados para ser aprovechada efectivamente.

La elección de métricas MAE y RMSE proporciona una evaluación comprehensiva del rendimiento, donde MAE ofrece una interpretación directa del error promedio en kilogramos, mientras que RMSE penaliza más severamente los errores grandes, proporcionando información sobre la robustez del modelo ante *outliers*.

Esta investigación establece una base sólida para la predicción de ventas de

productos agrícolas, identificando qué tipos de información son más valiosos y qué arquitecturas de modelos son más efectivas para diferentes configuraciones de datos.

Integración en la Aplicación

Los resultados obtenidos en la investigación no se limitan a un análisis teórico, sino que se han incorporado directamente en la aplicación web desarrollada. La integración de los modelos de predicción permite que el usuario final disponga de una herramienta práctica para la toma de decisiones en el negocio.

Concretamente, el *backend* expone un servicio que ejecuta los modelos entrenados y devuelve las predicciones de ventas en formato estructurado. El *frontend*, por su parte, muestra estas predicciones junto con los valores históricos de ventas en gráficos interactivos que facilitan la comparación entre lo real y lo estimado. Además, se presentan métricas de error (MAE y RMSE) asociadas al modelo seleccionado, lo que aporta transparencia sobre la fiabilidad de las predicciones.

De este modo, la aplicación no solo ofrece una visualización descriptiva de las ventas pasadas, sino que también incorpora un componente predictivo que permite anticipar la demanda futura. Esto constituye un apoyo tangible para la planificación de compras, la gestión de inventario y la toma de decisiones estratégicas en el negocio.

Conclusiones

El objetivo era ver qué modelo compensa más en la práctica: uno *eficaz* (menos error) o uno *eficiente* (más fácil y barato de entrenar y mantener). La comparación se hizo entre regresión y LSTM usando MAE/RMSE como guía.

Resultados obtenidos. Con la serie sin variables adicionales (solo ventas), la regresión sigue siendo la referencia: el LSTM **empeora** el error en **+6,9 % (MAE)** y **+3,9 % (RMSE)**. Cuando se incorpora información que introduce relaciones más complejas o con desfase temporal, el LSTM **sí mejora** de forma clara: con la variable **precio** reduce el error frente a regresión en **-25,1 % (MAE)** y **-22,4 % (RMSE)**; con la variable **calendario** mejora frente al mejor modelo clásico en **-17,3 % (MAE)** y **-14,8 % (RMSE)**.

En resumen, cuando se utiliza únicamente la serie de ventas, el modelo de regresión ofrece un mejor rendimiento; en cambio, al enriquecer la serie con variables adicionales, el modelo LSTM muestra un desempeño superior.

Coste de la mejora obtenida La regresión entrena en segundos, casi sin ajustes y con resultados muy estables; además es fácil de explicar. El LSTM necesita más tiempo de entrenamiento, más decisiones (tamaño de la red, regularización, tasa de

aprendizaje) y un poco más de cuidado en despliegue. Todo ello añade trabajo de operación y de seguimiento del modelo.

Cómo elegir con criterio. Si se busca **simplicidad y estabilidad** para una serie con patrones regulares, elegir **regresión** es lo más sensato. Si se dispone de **señales útiles** (precio, calendario) y se acepta un mayor coste operativo a cambio de **menos error**, el LSTM es la apuesta adecuada. Una regla práctica: partir de regresión como línea base y pasar a LSTM sólo cuando las pruebas muestren mejoras sostenidas por encima de **10–15 %** en MAE.

Cierre. Las pruebas apuntan a una idea simple: *la complejidad del modelo debe crecer sólo cuando los datos lo piden*. Con la serie de ventas sin más, la regresión ofrece un equilibrio excelente entre error y esfuerzo. Al sumar señales que añaden no linealidad o desfases, el LSTM aprovecha mejor la información y reduce el error en porcentajes que pueden justificar el esfuerzo extra.

3.2. Clustering

Objetivo y alcance del *clustering*

El propósito de esta fase es **identificar patrones de co-ocurrencia de compra** a partir de las cestas registradas, con tres beneficios directos para el negocio: (i) recomendaciones cruzadas de productos en la aplicación, (ii) mejor planificación de inventario por familias que suelen demandarse conjuntamente y (iii) diseño de *packs* y promociones. El enfoque se apoya en *K-Means* aplicado a una matriz transacción–producto y se valida con la regla del codo y una inspección visual mediante t-SNE.

Preparación de datos y definición de cesta

Se parte del conjunto de ventas depurado (véase la sección de EDA) y se aplican los siguientes pasos:

1. **Filtrado de devoluciones:** se retienen únicamente las filas con `Sale or Return = 'sale'`.
2. **Marca temporal:** se construye `Datetime` combinando `Date` y `Time`.
3. **Identificador de compra:** se define `transaction_id` como el redondeo a minuto de `Datetime`; todas las líneas del mismo minuto se consideran una misma cesta.¹
4. **Cestas unitarias:** se eliminan cestas con un solo producto para centrarse en co-ocurrencias informativas.

¹Este criterio se ajusta al flujo de caja del negocio y reduce el *over-splitting* de tickets.